

WORKSHOP · 8. SEPTEMBER 2026

Local-First AI

Alles läuft auf deinem Laptop.



WLAN? Brauchen wir nicht.



Ladekabel einstecken – sechs Stunden Modelle rechnen zieht Strom.



WER DA VORNE STEHT

David Pinezich

CEO & Gründer apigenio GmbH, Wohlen AG · Solution Architect

WAS ICH MACHE

15 Jahre Software und IT-Architektur: PSI, Architonic, ti&m, Helsana. Heute Architektur, Cloud und Workshops für KMU.

WARUM LOKAL

Sicherheit ist mein Fach (CISM). Daten, die das Haus nicht verlassen müssen, sollen es auch nicht.

NEBENBEI

Seit 2017 Python-Dozent an der Uni Zürich, vier Jahre Informatiklehrer an der Kanti Baden. «Make IT simple. Make it human.»

Vier Befehle, dann ist dein Laptop bereit.

1 `ollama list`

Siehst du `qwen3.5:9b` und `embeddinggemma` ? Sonst: `ollama pull qwen3.5:9b` . Und Hand hoch.

2 `git clone https://github.com/dpinezich/workshop-local-first-ai local-first-ai`
`cd local-first-ai`

Kein git? Auf github.com/dpinezich/workshop-local-first-ai «Code → Download ZIP», entpacken, Ordner in `local-first-ai` umbenennen.

3 `./verify-setup.sh` # Windows: `.\verify-setup.ps1`

Gelb oder rot? Hand hoch – Instruktor kommt.

4 `ollama run qwen3.5:9b --think=false "Hallo"`

Lädt 6 GB in den RAM – die erste Antwort dauert, ab dann geht's schnell. Antwortet es und kehrt zum Prompt zurück? Bereit. (Mit Frage als Argument beendet sich `ollama run` von selbst – `/bye` braucht es nur im Chat-Modus ohne Argument.)

AB JETZT AHA #1

WLAN aus.

Bis 16:30 braucht dein Laptop kein Internet mehr.

Chat, Bilder, Transkription, Suche, dein eigenes UI – alles rechnet hier. Wer's nicht glaubt:
ausprobieren.

AUFWÄRMEN • QUIZ RUNDE 1

Stimmt's?

Sechs Behauptungen. Hand hoch, wenn ihr glaubt: **stimmt**. Hand unten: **stimmt nicht**.

Richtig geraten = 1 Punkt. Die Antworten kommen im Lauf des Tages – jede Behauptung gehört zu einem Kapitel.

**«Ein Sprachmodell schlägt
seine Antworten in
einer Datenbank nach.»**

**«Ohne Internet kann ein
Sprachmodell nicht antworten.»**

**«Ein Modell mit 9 Milliarden
Parametern braucht
mindestens 32 GB RAM.»**

**«Für einen deutschen Text
braucht das Modell mehr
Tokens als für denselben
Text auf Englisch.»**

**«Das Modell merkt sich,
was ihr ihm vor fünf
Minuten geschrieben habt.»**

**«Bei einer Klassifikationsaufgabe
erkennt man sofort,
ob der Laptop oder die
Cloud geantwortet hat.»**

EINE WOCHE BEI MUSTER SOLUTIONS

Mo

Di

Mi

Do

Fr

KAPITEL 1

Zwei Mails

Warum lokal · Was ein Modell ist

Aha #1 · Flugmodus

Zwei Mails, bevor der Kaffee fertig ist

SI Von: Dr. Sara Imhof • Datenschutz, Kantonsspital (Kunde)

Betreff: Ihr «AI-Assistent» – wohin gehen unsere Daten?

«Ihre Leute fassen unsere Tickets damit zusammen.
Bitte bestätigen Sie schriftlich, dass keine
Patientendaten an Dritte übermittelt werden.»

RH Von: Reto Hess • CFO

Betreff: API-Rechnung August

«CHF 4'200. Im Januar waren es 800. Wächst das so
weiter? Ich brauche bis Freitag eine Alternative
oder eine Begründung.»

Lea Brunner, Leiterin IT, 60 Mitarbeitende, Muri AG. Das Kantonsspital ist Kunde – seine Tickets laufen durch Leas Assistenten. Sie hat bis Freitag. **Ihr seid ihr Team.**

Drei Gründe. Lea hat gerade alle drei auf dem Tisch.

1



Compliance

Saras Frage.
Patientendaten dürfen das Haus nicht verlassen. Kein DPA der Welt ersetzt «es geht gar nicht raus».

2



Kosten

Retos Rechnung. Bei hohem Volumen einfacher Aufgaben summieren sich Cents zu Tausendern.

3



Verständnis

Leas eigener Grund: Wer es lokal betreibt, sieht zum ersten Mal, was ein Modell wirklich ist.

Ein Modell ist eine Datei voller Zahlen, die das nächste Wort rät



Token

Die Einheit, in der das Modell liest und schreibt. Wird auch abgerechnet.

Parameter

Die gelernten Zahlen. «9B» = 9 Milliarden. Mehr = klüger, aber grösser und langsamer.

Inferenz

Das Anwenden (nicht das Trainieren). Nur das passiert heute auf eurem Laptop.

Der Tokenizer ist eine feste Tabelle mit 150'000 Einträgen

TEXT	TOKENS	UNGEFÄHR SO ZERLEGT
Internet	1	Internet
Rechnung	3	Rech · n · ung
Ferienregelung	4	Fer · ien · regel · ung
Chuchichäschtli	6	Chu · chi · chäsch · tli ...
Wie viele Tage Urlaub habe ich?	7	≈ ein Token pro Wort

Häufiges = ein Token

«Internet» kommt im Trainingstext millionenfach vor und bekommt einen eigenen Eintrag. «Rechnung» nicht – Deutsch kostet rund 1.5× mehr Tokens als Englisch.

Warum das zählt

Tokens sind die Währung: Kontextfenster, Tempo und Cloud-Preise werden in Tokens gerechnet. Und Dialekt zerfällt in Krümel – das Modell hat «Chuchichäschtli» nie als Ganzes gesehen.

Gemessen mit qwen3.5:9b am 3. September. Zerlegung ist vereinfacht dargestellt.

Acht Kapitel. Sieben Aha-Momente. Ein Quiz.

VORMITTAG • 9:00–12:15

Zwei Mails

#1

Flugmodus-Demo • Drei Gründe
• Quiz Runde 1

9:00

Die Einkaufsliste

#2

Modelle • Motorhaube •
Quantisierung • Runtimes

9:37

Der Klassifikator

#3

Eine Zeile Code • Copilot offline
10:30

Sitzung & Rechnung

#4

Whisper • Vision • Quiz Runde 2
11:22

NACHMITTAG • 13:15–16:30

Jonas fragt

#5

Embeddings • Urlaub findet
Ferien • RAG

13:15

Chat für alle

#6

Der Stack • Router • euer UI
14:08

Die GL-Sitzung

#7

Blind-Test-Finale • Matrix •
Kosten
15:30

Der Rollout

Diskussion • Epilog • Take-Home
15:46

Fünf Bausteine, ein Reality-Check. Der Vertrag für heute.

DER STACK

- | | | |
|---|----------------------------|--------|
| 1 | Chat-Modell | Kap. 3 |
| 2 | Embeddings + RAG | Kap. 5 |
| 3 | Whisper · Transkription | Kap. 4 |
| 4 | Vision · Bilder | Kap. 4 |
| 5 | Python-Backend mit Routing | Kap. 6 |

DER REALITY-CHECK

- | | |
|--------------------------------------|--------|
| Modelle, Quantisierung, Runtimes | Kap. 2 |
| Code-Assistenz offline | Kap. 3 |
| Eigene Benchmarks: lokal vs. Cloud | Kap. 7 |
| Intern verkaufen · im Team verteilen | Kap. 8 |
| Euer Stack mit Vue-Oberfläche | 16:30 |

Wo wir ein Versprechen einlösen, steht das Zitat unten auf der Folie. Am Schluss haken wir ab.

Blind-Quiz: lokal oder cloud?

- Eine Aufgabe aus Leas Alltag, zwei Antworten: A und B
- Eine vom Laptop (qwen3.5:9b), eine von Claude aus der Cloud
- Ihr ratet per Handzeichen, welche **lokal** ist
- Richtig = 1 Punkt. Tafel. Drei Runden. Schoggi.

Runde 1 • erledigt

Die sechs Behauptungen von vorhin

Runde 2 • vor dem Mittag

3 Paare • A oder B

Finale • am Nachmittag

8 Paare • Leas Beweis für die GL

EINE WOCHE BEI MUSTER SOLUTIONS

Mo

Di

Mi

Do

Fr

KAPITEL 2

Die Einkaufsliste

Modelle 2026 · Motorhaube · Quantisierung · Runtimes

Aha #2 · JPEG für Gewichte

Montag, 10 Uhr. Lea macht Inventur.

Was das Team hat

- 37 MacBooks, 16 GB, Apple Silicon
- 14 Windows-Laptops, davon 3 mit Nvidia-GPU
- Ein alter Intel-Mac in der Buchhaltung
- Ein Server im Keller: 128 GB RAM, keine GPU

Was sie wissen muss

- Welches Modell läuft auf 16 GB und ist gut genug auf Deutsch?
- Was tun mit dem Intel-Mac?
- Warum ist das Modell «Q4»? Was heisst das?
- Ollama, llama.cpp, vLLM – was davon?

Genau eure Situation im Raum. Also: die Theorie dahinter, dann der Entscheidungsbaum.

BEGRIFFE IN DIESEM KAPITEL

Transformer

Attention

MoE

Quantisierung

GGUF

KV-Cache

Bandbreite

Prefill / Decode

Runtime

Ollama

Wie viel RAM? Wie schnell?

Formel 1 • Speicher

$\text{RAM} \approx \text{Parameter} \times \text{Bytes pro Parameter}$

9B × 2 Byte (16 Bit) $\approx 18 \text{ GB}$ **passt nicht in 16 GB**

9B × 1 Byte (Q8) $\approx 11 \text{ GB}$ **knapp**

9B × ½ Byte (Q4) $\approx 6.6 \text{ GB}$ **passt, mit Luft**

Plus Kontext: je länger das Gespräch, desto mehr – warum das RAM kostet, in zehn Minuten. Faustregel bei normalem Chat: Modell + 2 GB. Reale Dateien liegen 20–30 % über der Formel: Q4_K_M sind ~4.8 Bit, Ausgabeschicht und Embeddings bleiben genauer, bei qwen3.5 ist der Vision-Encoder mit drin.

Formel 2 • Tempo

$\text{Token/s} \approx \text{Speicherbandbreite} \div \text{Modellgrösse}$

M1 Pro • 200 GB/s $\div 6.6 \text{ GB} \approx 30 \text{ Token/s}$ **gemessen: 21**

M4 Max • 546 GB/s $\div 6.6 \text{ GB} \approx 80 \text{ Token/s}$

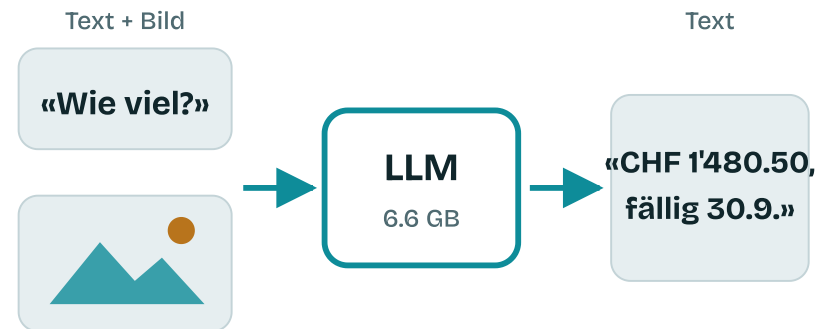
Laptop-CPU • 50 GB/s $\div 6.6 \text{ GB} \approx 7 \text{ Token/s}$

Pro Token wird das ganze Modell einmal durch den Speicher geschoben – darum zählt Bandbreite, nicht Rechenleistung. Real: 70–80 % der Formel.

Darum Apple Silicon (Unified Memory), darum Quantisierung, darum «kleiner ist oft besser».

Was rein geht, was raus kommt

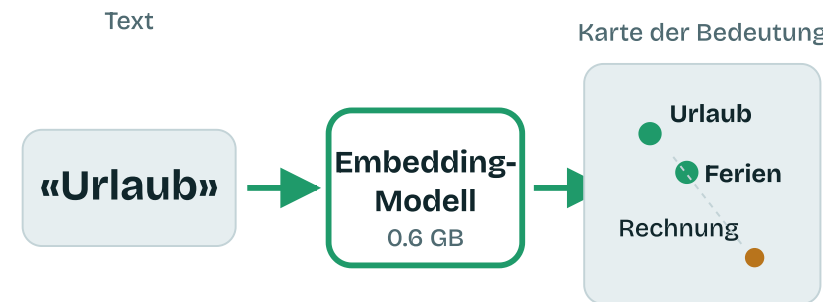
Chat + Vision



Das Modell, mit dem du redest. Liest Text und Bilder, schreibt Text. Chat, Zusammenfassen, Klassifizieren, Rechnung lesen.

qwen3.5:9b

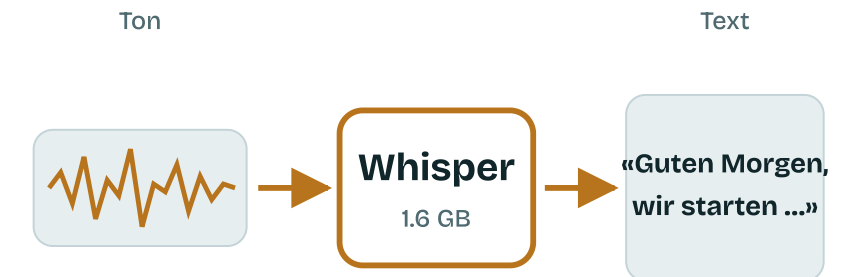
Embeddings



Das LLM kennt euer Wiki nicht, und die Volltextsuche findet «Urlaub» nicht, wenn dort «Ferien» steht. Ein Embedding ist eine Adresse auf der Karte der Bedeutung: «Urlaub» liegt neben «Ferien», weit weg von «Rechnung». Suchen heisst Nachbarn finden – so kommen die richtigen 3 von 200 Seiten zum LLM.

embeddinggemma

Transkription



Hört zu, schreibt mit. Kein Ollama-Modell, sondern ein eigenes Programm (whisper.cpp) mit eigener Modelldatei.

whisper large-v3-turbo

Welche Hardware? Was willst du tun? Dann steht das Modell.

HARDWARE	CHAT + VISION	EMBEDDINGS	TRANSKRIPTION
8 GB • CPU-only Leas Intel-Mac	qwen3.5:4b 3.4 GB	embeddinggemma	whisper small
16 GB • Apple Silicon / GPU ← heute alle hier	qwen3.5:9b 6.6 GB oder gemma4:e4b-it-qat	embeddinggemma	whisper large-v3-turbo
32 GB+ / Leas Server	qwen3.5:27b • gemma4:12b qwen3.6:27b	bge-m3 stark auf DE	whisper large-v3-turbo
Server mit GPU	vLLM statt Ollama • 120B-Modelle • 50 Leute gleichzeitig → Kapitel 8		

Tag ohne Suffix = Q4_K_M. Stand 2. September 2026 – in drei Monaten selbst nachschauen.

AUSSCHREIBUNG «Welche Modelle 2026 lokal wirklich taugen, wie man sie effizient betreibt, wo die Qualität überrascht und wo sie nicht reicht.»

Alle offen, alle auf Ollama. Der Charakter ist verschieden.

QWEN · ALIBABA

Der Streber

Jede Grösse, jede Sprache, Vision und Denken eingebaut. Apache 2.0.

qwen3.5:9b · heute

16 GB RAM GPU empfohlen

GEMMA · GOOGLE

Der Allrounder

Kleiner Bruder von Gemini. Viel Qualität pro GB, QAT-Varianten. Eigene Lizenz.

gemma4:e4b-it-qat

16 GB RAM GPU empfohlen

LLAMA · META

Der Klassiker

Hat 2023 die Bewegung gestartet. Grösste Community, meiste Fine-Tunes. Eigene Lizenz.

llama3.1:8b · 70b: 48 GB

16 GB RAM GPU empfohlen

MISTRAL · PARIS

Der Europäer

Das Datenschutz-Argument im Meeting. Kompakte Modelle, Apache 2.0.

mistral-small3.2:24b

32 GB RAM GPU Pflicht

PHI · MICROSOFT

Klein und überraschend

Auf Lehrbuch-Daten trainiert. 4B schlägt manches 8B. MIT-Lizenz.

phi4-mini

8 GB RAM CPU reicht

DEEPSEEK · HANGZHOU

Der Denker

R1 hat das Denken per RL gelernt – seither denken alle. Gross: 671B MoE. MIT.

deepseek-r1:8b · 671b: Server

16 GB RAM GPU empfohlen

GPT-OSS · OPENAI

Der Spätzünder

OpenAIs erstes offenes Modell seit GPT-2. MoE 20B / 120B, Apache 2.0.

gpt-oss:20b · 16 GB knapp

24 GB RAM GPU Pflicht

SPEZIALISTEN

Ein Job, sehr gut

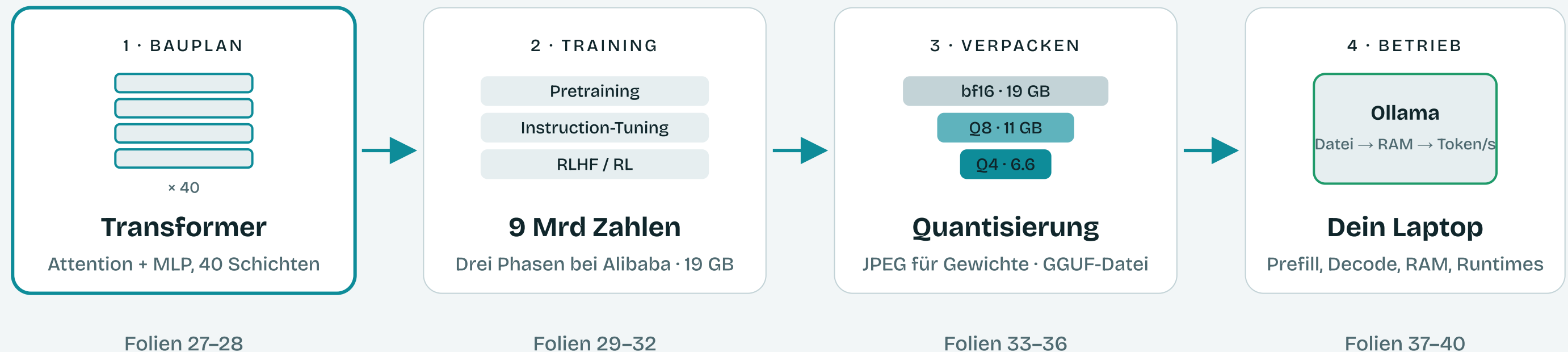
OCR, Code, Vision auf Mini-Hardware, Sprache zu Text.

glm-ocr · moondream · whisper
· qwen3-coder (32 GB)

8 GB RAM CPU reicht

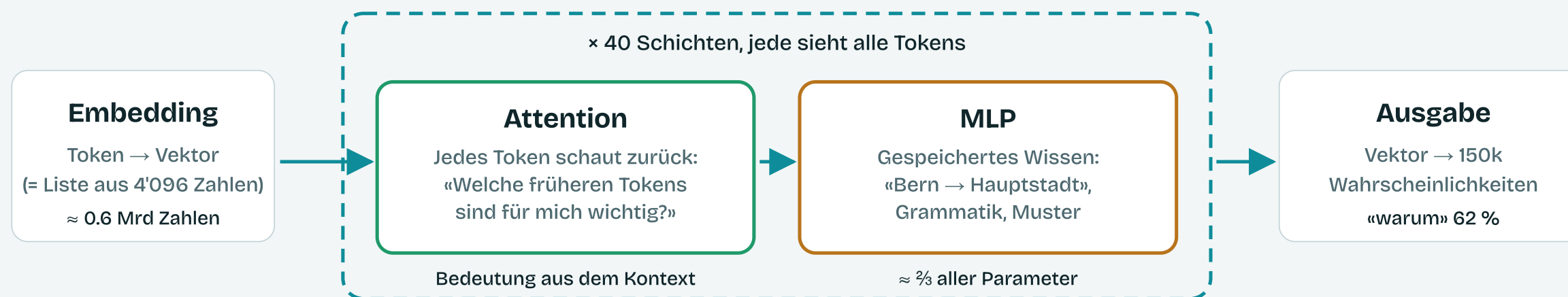
Stand September 2026. Die Namen bleiben, die Versionsnummern nicht. Die lange Liste mit Grösse, Stärke und Lizenz: docs/modelle-2026.md im Repo.

Der Weg eines Modells: vom Bauplan auf deinen Laptop



Vier Stationen, in dieser Reihenfolge. Oben rechts auf jeder Folie steht, wo wir gerade sind.

Ein Transformer: Nachschlagen, Umschauen, Wissen abrufen – 40-mal hintereinander



Der KV-Cache aus Block 2 = Schlüssel und Werte (Key, Value) aller bisherigen Tokens. Deshalb kostet Kontext RAM.

Attention = Umschauen

Die

Bank

am

Fluss

Das Token «Bank» holt sich seine Bedeutung von den Nachbarn. 2017 erfunden, seither die Basis von allem.

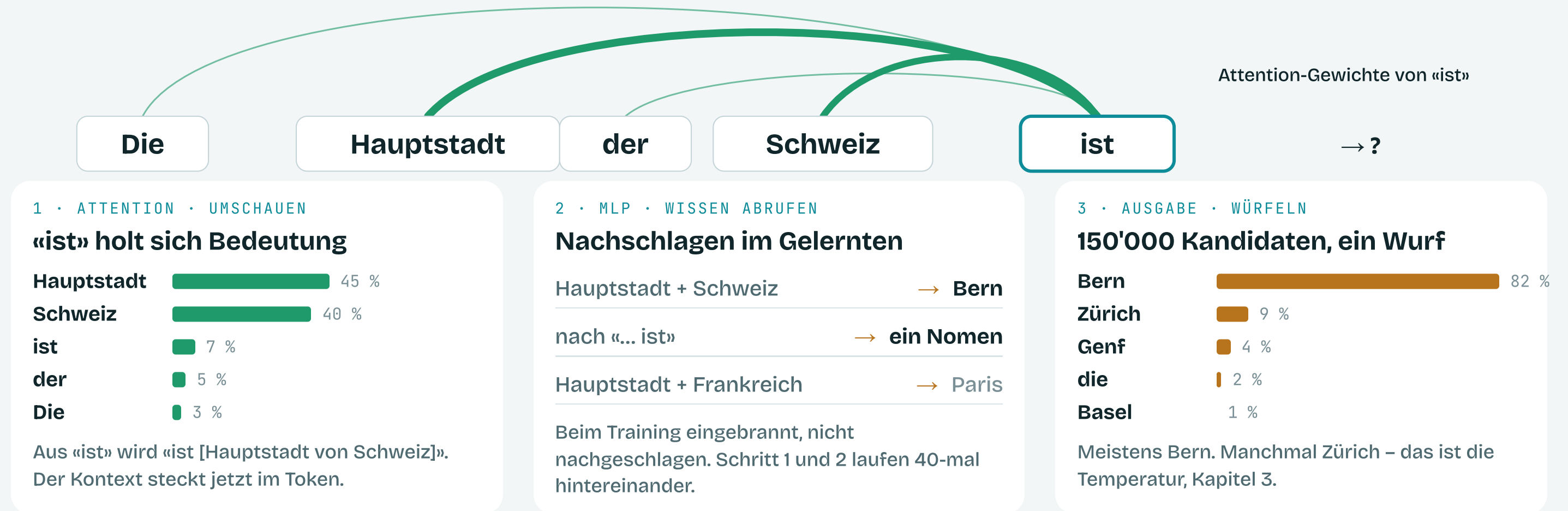
MLP = Nachschlagen

Zwei bis drei Matrizen pro Schicht, hier sitzt das Faktenwissen. Der grösste Teil der 9 Milliarden – deshalb bringt Quantisieren hier am meisten.

40-mal, dann raten

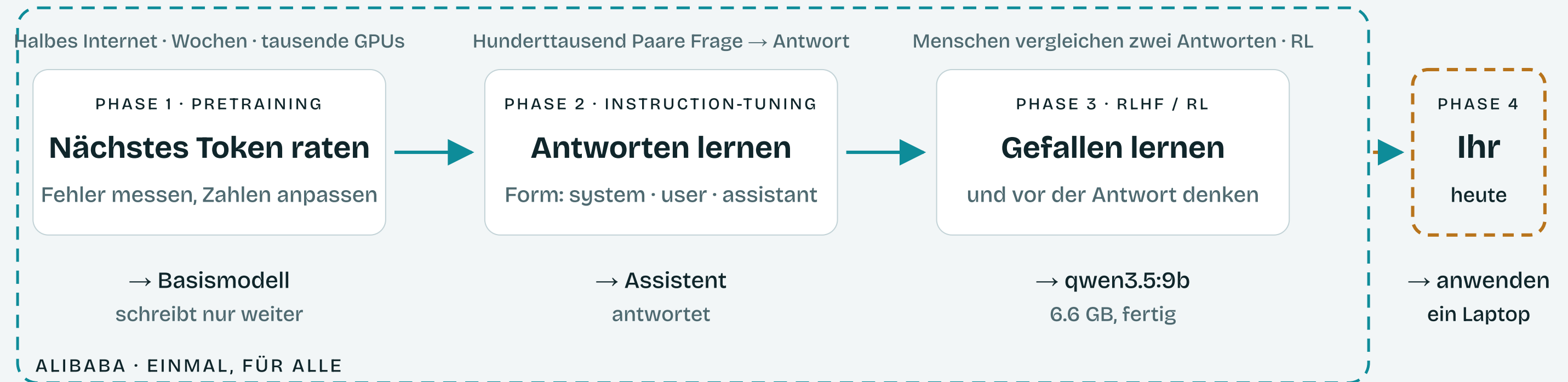
Grob: frühe Schichten erkennen Wortarten, mittlere Bedeutung, späte formulieren. Dann ein Würfelwurf über 150'000 Kandidaten.

«Die Hauptstadt der Schweiz ist» – und jetzt?



«... ist **Bern**». Anhängen, und alles nochmal für das nächste Token. Zahlen illustrativ.

Drei Phasen Training. Ihr macht heute die vierte: anwenden.



Alle drei Phasen hat Alibaba gemacht. Ihr wendet das Ergebnis an – deshalb reicht ein Laptop.

Nächstes Token raten, billionenfach

Die Aufgabe

- Text aus dem halben Internet
- Nächstes Token vorhersagen, Fehler messen
- Zahlen ein winziges bisschen anpassen
- Billionen Mal. Wochen auf tausenden GPUs.

Das Ergebnis: ein Basismodell

Eingabe

Wie heisst die Hauptstadt der Schweiz?

Basismodell schreibt weiter ...

Wie heisst die Hauptstadt von Österreich?

Frage 3: Welcher Fluss fliesst durch Basel?

Es beantwortet Fragen nicht – es schreibt sie weiter.
Nichts anderes hat es gelernt.

Was daraus folgt

Das «Wissen» ist ein Nebenprodukt: statistisch komprimiert, nicht nachgeschlagen. Warum das zu Halluzinationen führt – Kapitel 5.

Antworten lernen. Dann Gefallen lernen.

PHASE 2 • INSTRUCTION-TUNING

Aus Beispielen antworten lernen

```
system:    Du bist ein Assistent.  
user:      Hauptstadt der Schweiz?  
assistant: Bern.
```

Ein paar hunderttausend solcher Paare, von Menschen geschrieben. Dieselbe Rechnung wie in Phase 1 – jetzt lernt es die Form. Ab hier ist es ein Assistent.

PHASE 3 • RLHF / RL

Gefallen und Denken lernen

```
Antwort A: Bern.                ✓ besser  
Antwort B: Die Schweiz hat keine  
Hauptstadt im engeren Sinn ...
```

Menschen vergleichen zwei Antworten, das Modell wird auf die bessere gezogen. Mit Reinforcement Learning lernt es, vor der Antwort «zu denken» – der Denk-Modus von Qwen, R1, Claude.

Phase 4 • was ihr heute macht

Das fertige Modell anwenden. Alle drei Phasen hat Alibaba gemacht. Deshalb reicht ein Laptop.

Alle raten das nächste Token. Der Unterschied liegt auf drei Achsen.

MODELL	ARCHITEKTUR	TRAINING	BETRIEB
Qwen 3.5 · 9B	Dicht. Vision eingebaut, Denken zuschaltbar.	Offen (Apache 2.0), 201 Sprachen.	Euer Laptop, 6.6 GB
Llama · Gemma · Mistral	Dicht. Attention-Varianten für kleineren KV-Cache.	Offen.	Laptop bis Server
DeepSeek V3 / R1	Mixture of Experts: 671B gespeichert, 37B rechnen pro Token.	Offen. R1 hat das «Denken» per RL gelernt – der Auslöser für alle Denk-Modi.	Server
GPT-OSS	Mixture of Experts, 20B / 120B.	Offen (OpenAI).	20B auf 16 GB
Claude	Unveröffentlicht. Frontier-Grösse, 1M Kontext.	Geschlossen. Mehr Daten, mehr Rechenzeit, aufwendiges Post-Training.	Nur Cloud

Claude «besser»? Kein anderer Trick – mehr von allem. Beim Erkennen holt ein 9B-Modell auf, beim Schlussfolgern nicht.

AUSSCHREIBUNG «Die Reife der offenen Modelle macht lokales AI erstmals alltagstauglich: Qwen, Llama, Phi, Gemma, Mistral liefern in 3B–14B Parametern erstaunliche Qualität.»

Quantisierung ist JPEG für Gewichte

Muster Elektro AG

Bahnhofstrasse 12, 5630 Muri AG

RECHNUNG

Rechnungsnummer: 2026-0815

Datum: 14.08.2026

Kunde: Beispiel GmbH, Zuerich

Bezeichnung	Menge	Einzelpreis	Betrag
Installationsarbeiten	6 h	CHF 140.00	CHF 840.00
LED-Panel 60x60	4 Stk	CHF 89.50	CHF 358.00
Kleinmaterial	1 Psch	CHF 45.00	CHF 45.00
Anfahrt	1 Psch	CHF 60.00	CHF 60.00

PNG • 44 KB •
verlustfrei

Muster Elektro AG

Bahnhofstrasse 12, 5630 Muri AG

RECHNUNG

Rechnungsnummer: 2026-0815

Datum: 14.08.2026

Kunde: Beispiel GmbH, Zuerich

Bezeichnung	Menge	Einzelpreis	Betrag
Installationsarbeiten	6 h	CHF 140.00	CHF 840.00
LED-Panel 60x60	4 Stk	CHF 89.50	CHF 358.00
Kleinmaterial	1 Psch	CHF 45.00	CHF 45.00
Anfahrt	1 Psch	CHF 60.00	CHF 60.00

JPEG 60 % • 20 KB •
gerundet

Dasselbe Modell, drei Grössen

bf16

19 GB

Q8_0

11 GB

Q4_K_M

6.6 GB

← Ollama-Default

Gleich live: Q4 gegen Q8, gleiche Frage, `ollama ps` daneben.

Dieselbe Rechnung, halb so gross – seht ihr den Unterschied? Genau so werden die 9 Milliarden Zahlen von 16 auf 4 Bit gerundet. Formel 1: 18 GB → 6.6 GB.

Ein Skalierungsfaktor pro Block

So funktioniert Q4

32 Gewichte (16 Bit): 0.0213 -0.0187 0.0091 ...
 ↓ ein gemeinsamer Massstab $s = 0.0014$
32 Integer (4 Bit): 15 -13 6 ... ← -8...+7
Zurückrechnen: $w \approx s \times q$ Fehler klein, weil Block klein

Pro Block ein Massstab statt pro Zahl 16 Bit. 4.5 Bit pro Gewicht im Schnitt – ein Viertel des Speichers. Die «K_M»-Varianten geben einzelnen empfindlichen Tensoren 6 Bit, dem Rest 4.

Wie viel Qualität kostet es?

STUFE	GRÖSSE 9B	QUALITÄTSVERLUST
Q8_0	≈ 11 GB	praktisch keiner
Q4_K_M	≈ 6.6 GB	kaum messbar · Sweet Spot
Q3_K_M	≈ 5 GB	spürbar bei Reasoning
Q2_K	≈ 3.5 GB	deutlich, oft wirr

Gemessen als «Perplexity»: wie überrascht ist das Modell von echtem Text. Faustregel: lieber das grössere Modell in Q4 als das kleinere in Q8.

Vom 18-GB-Original zur 6.6-GB-Datei auf deinem Laptop

1 • ORIGINAL

Alibaba • Hugging Face

Das Modell, wie es das Training verlässt: BF16, rund 18 GB. Läuft nur auf 24 GB RAM oder mehr.

```
Qwen/Qwen3.5-9B
```

2 • QUANTISIEREN

llama.cpp • llama-quantize

Ein Kommandozeilen-Werkzeug rechnet Block für Block auf 4 Bit um. Auf dem Laptop rund 10 Minuten.

```
llama-quantize in.gguf  
out.gguf Q4_K_M
```

3 • VERTEILEN

GGUF in der Ollama-Library

Eine Datei, alles drin: Gewichte, Tokenizer, Chat-Vorlage. Tag ohne Suffix ist immer die Q4_K_M.

```
ollama pull qwen3.5:9b → 6.6  
GB
```

4 • DER NAME

Q4_K_M

Q quantisiert • 4 Bit • K K-Quants, das Blockverfahren von eben • M Medium: S / M / L regeln, wie viele empfindliche Tensoren mehr Bit bekommen.

```
qwen3.5:9b-q8_0 • die Demo  
von vorhin
```

Wer quantisiert? Ollama selbst, der Hersteller, oder die Community auf Hugging Face (Unsloth, Bartowski). Gleiche Stufe, gleiches Verfahren – die Dateien sind praktisch identisch.

GGUF, GPTQ, AWQ, MLX

GGUF Das Format von llama.cpp und Ollama. CPU + GPU, überall.

Das werdet ihr sehen. Immer.

GPTQ GPU-Quantisierung mit Kalibrierungsdaten.

Erst mit Server relevant

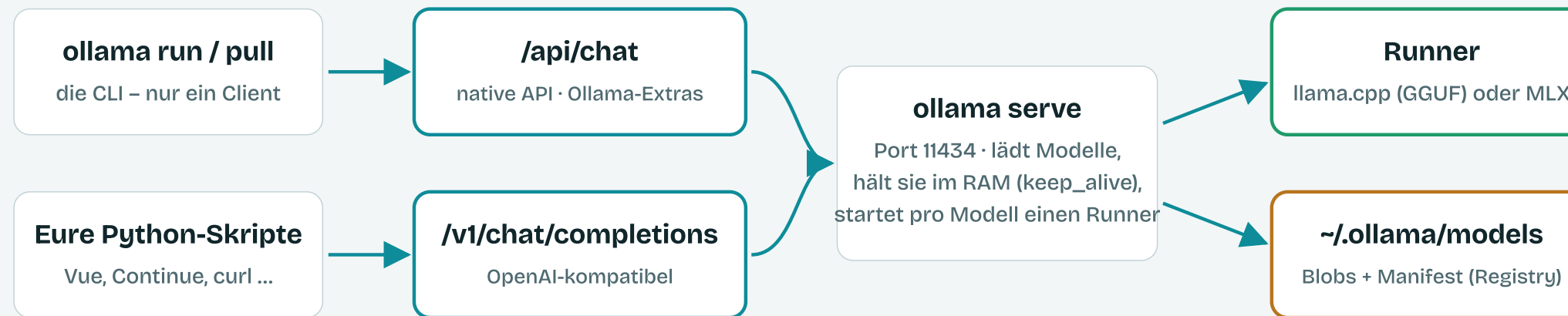
AWQ Schützt die wichtigen Gewichte. Schnell auf GPU.

Erst mit Server relevant

MLX Apples Format. In Ollama als `-mlx`-Tag.

Zuhause: `qwen3.5:9b-mlx`

Ein Server, eine CLI, eine Registry, zwei APIs



Deshalb «eine Zeile»

Die zweite API spricht OpenAI-Dialekt. Jedes Tool, das OpenAI kann, kann Ollama.

Deshalb kopierbar

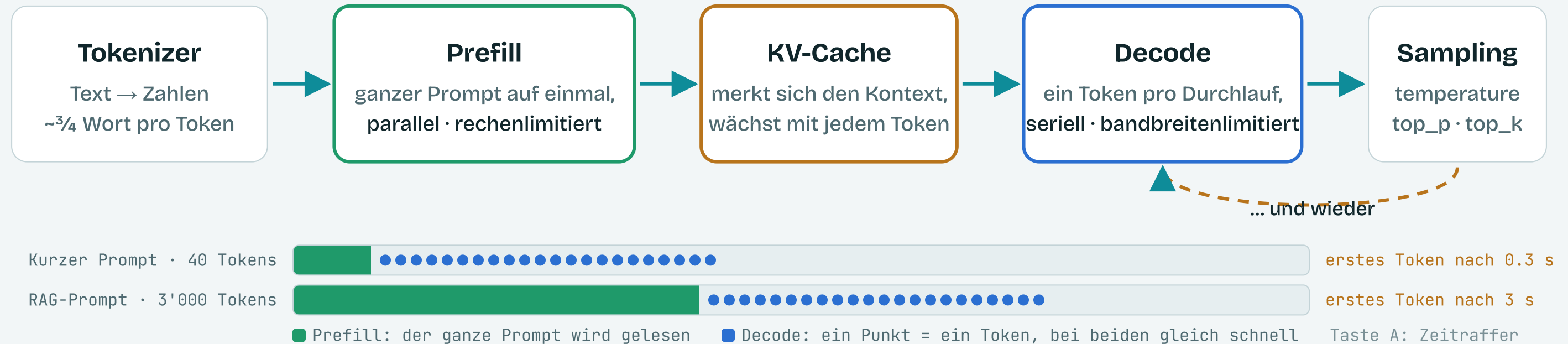
Der Modellordner ist nur Dateien – per AirDrop weitergeben.

`OLLAMA_HOST=0.0.0.0`
macht den Server im Netz erreichbar (Kapitel 8).

Deshalb `ollama ps`

Zeigt, welcher Runner gerade läuft, wie viel RAM er hält und wann er entladen wird.

Prefill ist schnell, Decode ist der Flaschenhals



Time to first token

= Prefill. Langer Prompt → spürbare Pause, dann läuft es.

Token pro Sekunde

= Decode. Formel 2: Bandbreite durch Modellgrösse.

Warum vLLM auf dem Server

Decode nutzt die GPU zu 10 %. vLLM legt 50 Nutzer in einen Durchlauf. Auf dem Laptop bringt das nichts.

Womit lässt man das Modell laufen?

heute: Ollama

RUNTIME	WAS ES IST	WANN LEA ES NIMMT
Ollama	Server + CLI + Modell-Registry. Ein Installer, ein Befehl, läuft. Nutzt intern llama.cpp oder MLX.	Für alle 52 Laptops. 95 % aller Fälle.
llama.cpp	Die C++-Engine unter Ollama, direkt bedient. Jede Einstellung von Hand: wie viele Schichten auf die GPU, wie viele CPU-Threads, wie gross der Kontext, welches Sampling.	Für den Server im Keller ohne GPU, wenn Ollamas Automatik nicht das Beste rausholt. Oder zum Lernen.
MLX	Apples eigenes Framework für Apple Silicon. In Ollama als <code>-mlx</code> -Tags, sonst über <code>mlx-lm</code> .	Bonus auf den MacBooks: oft 10–30 % schneller. Kein Muss.
vLLM	Server-Runtime für Nvidia-GPUs. Bedient viele Nutzer gleichzeitig (Continuous Batching, Folie vorher).	Erst wenn der Server eine GPU bekommt und 50 Leute gleichzeitig fragen. Auf dem Laptop nutzlos.

Drei Befehle, drei Zahlen: Was steckt in deinem Ollama?

Ziel: Parameter, Quantisierung, RAM und Token/s einmal selbst ablesen und mit Folie 22 vergleichen.

```
ollama show qwen3.5:9b
→ parameters 9.7B · quantization Q4_K_M · context 262144

ollama run qwen3.5:9b --think=false --verbose \
  "Was ist die Hauptstadt der Schweiz?"
→ prompt eval rate = Prefill · eval rate = Decode · Token/s

ollama ps
→ SIZE = Modell im RAM · PROCESSOR = GPU oder CPU?
```

Wer fertig ist, macht Pause. Windows: dieselben Befehle in PowerShell, den Befehl auf eine Zeile.

1 • Formel 1 • RAM

SIZE in `ollama ps` : mehr oder weniger als die 6.6 GB der Datei? Beides hat einen Grund.

2 • Formel 2 • Tempo

$\text{eval rate} = \text{eure Token/s} \cdot \text{Bandbreite eures Chips} \div 6.6 \text{ GB}$ – wie nah seid ihr dran? Mit dem Nachbarn vergleichen: M1, M3, Windows.

3 • Prefill gegen Decode

Nochmals mit langem Prompt: `"Fasse zusammen: $(cat docs/glossar.md)"`. Wie viel schneller ist `prompt eval rate` als `eval rate`?

Pause

Danach: Leas alter Ticket-Klassifikator und eine Zeile Code.

Drei Fragen an qwen3.5:9b: wo ist es gut, wo peinlich?

Ziel: Das Modell einmal selbst an seine Grenzen bringen, bevor wir es in Code einbauen.

```
ollama run qwen3.5:9b
>>> /set nothink      ← Denken aus
>>> ...eure Fragen...
>>> /bye
```

Je ein Beispiel merken: wo gut, wo peinlich. Wir sammeln sie gleich und brauchen sie um 15:30 wieder.

Antwort dauert 30 s? Dann `qwen3.5:4b` .

1 • Etwas Faktisches

«Wie hoch ist der Säntis?» Stimmt es? Wie sicher klingt es, wenn es falsch liegt?

2 • Etwas auf Schweizerdeutsch

«Antworte uf Schwiizerdütsch: Was isch es Embedding?» Wie weit kommt es?

3 • Eine Rechenaufgabe

«Zug ab Muri 14:07, Fahrzeit 38 Minuten – Ankunft?» Einmal ohne, einmal mit Denken (`/set think`).

EINE WOCHE BEI MUSTER SOLUTIONS

Mo

Di

Mi

Do

Fr

KAPITEL 3

Der Ticket-Klassifikator

Rollen und Kontext · die OpenAI-API · Copilot offline

Aha #3 · Eine Zeile Code

Dienstag. Ein Python-Skript, ein Jahr alt.

```
# tickets/klassifizieren.py - Auszug, geschrieben letzten Sommer
from openai import OpenAI
client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])

def kategorie(ticket_text):
    resp = client.chat.completions.create(
        model="gpt-...",
        messages=[{"role": "system", "content": PROMPT},
                  {"role": "user", "content": ticket_text}])
    return resp.choices[0].message.content
# ... läuft 1'400× am Tag. Jedes Ticket geht in die Cloud – auch die vom Spital.
```

Leas Frage

«Muss ich das neu schreiben? Die Devs haben keine Zeit. Und die wollen sowieso lieber einen Copilot.»

Die Antwort

Eine Zeile. Aber zuerst: was in diesem Skript eigentlich passiert.

Wer redet hier eigentlich mit wem?



localhost = «dieser Laptop» · Port = Türnummer · API = die vereinbarten Türen und Formate · Endpoint = eine davon

DIE ACHT WÖRTER

Client

Server

localhost

Port

API

Endpoint

JSON

Prompt

API · Endpoint

Die Schnittstelle: welche Adressen es gibt und wie Frage und Antwort aussehen. `/v1/chat/completions` ist ein Endpoint. «OpenAI-kompatibel» heisst: dieselben Adressen, dieselben Formate.

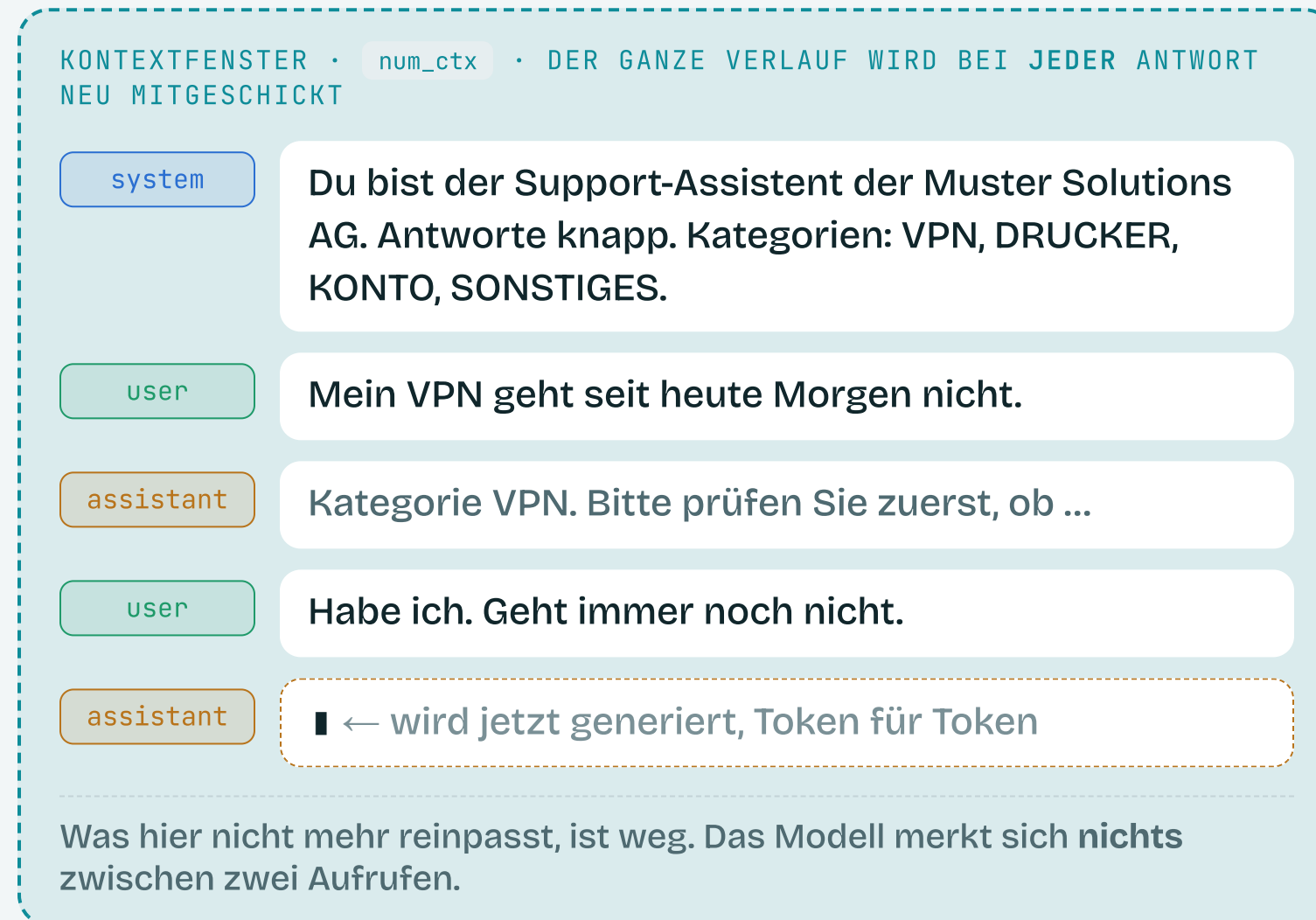
JSON

Das Textformat, in dem Programme Daten austauschen: geschweifte Klammern, Schlüssel, Werte. Lesbar für Mensch und Maschine. Die Rechnung in Kapitel 4 wird zu JSON.

Prompt

Alles, was ihr dem Modell schickt: Anweisung, Verlauf, Frage. Der **system-Prompt** ist der feste Teil davon – kommt gleich.

Drei Rollen, ein Fenster, kein Gedächtnis



system = die Stellenbeschreibung

Ton, Regeln, Wissen. Kommt immer zuerst. Hier steckt 80 % der Qualität, und bei RAG der Kontext.

user / **assistant** = das Protokoll

Der ganze Verlauf wird jedes Mal mitgeschickt. «Gedächtnis» ist nur: den Text wieder anhängen.

Kontextfenster = die Grenze

So viele Tokens passen rein. Mehr Kontext = mehr RAM (KV-Cache) und längeres Prefill.

Anatomie eines Ollama-Calls

```
$ ollama run qwen3.5:9b --think=false
```

- ① Modell laden: 6.6 GB → RAM (einmalig)
- ② Kontextfenster reservieren: num_ctx

```
>>> Was ist ein Embedding?
```

- ③ Chat-Template: system + Verlauf + Frage
- ④ Token für Token:
Wahrscheinlichkeit → Würfel → Wort

Ein Embedding ist ein Vektor, der die
Bedeutung eines Textes als Zahlen ...

- ⑤ Stop-Token → fertig · 21 Token/s (gemessen, M1 Pro)

temperature = der Würfel

0 = immer das wahrscheinlichste
Token, reproduzierbar. 1.5 = kreativ bis
wirr. Für Leas Klassifikator: 0.

num_ctx = das Fenster

Default 4'096. Für lange Dokumente
hochsetzen – kostet RAM.

Leas Klassifikator, wie er heute läuft

[tickets/klassifizieren.py](#) • Cloud

```
from openai import OpenAI
client = OpenAI(
    api_key="sk-...",          # der Schlüssel zur Cloud
)
resp = client.chat.completions.create(
    model="gpt-...",
    messages=[{"role": "system", "content": PROMPT},
               {"role": "user", "content": ticket}],
)
```

Wer hat so etwas schon einmal geschrieben? Genau dieser Code läuft in tausenden Firmen.

Eine Zeile. Der Rest bleibt.

tickets/klassifizieren.py • auf Leas Laptop

```
from openai import OpenAI
client = OpenAI(
    base_url="http://localhost:11434/v1", # DIE eine Zeile
    api_key="ollama", # Pflichtfeld, Wert egal
)
resp = client.chat.completions.create(
    model="qwen3.5:9b",
    messages=[{"role": "system", "content": PROMPT},
              {"role": "user", "content": ticket}],
)
```

Alles, was ihr über OpenAI-APIs wisst, gilt weiter. Und 1'400 Tickets am Tag verlassen das Haus nicht mehr.

Leas Klassifikator, auf deinem Laptop

Ziel: Ihr habt am Ende selbst einen lokalen API-Call geschrieben und gesehen, wie Streaming, System-Prompt und Temperatur wirken.

- 1. `01_erster_call.py` ausführen, dann die Frage ändern
- 2. `02_streaming.py` : den Tokens beim Entstehen zusehen
- 3. `03_system_prompt.py` : nur Schweizerdeutsch!
Dann `temperature 0` gegen 1.5
- 4. `04_code_assistent.md` , Demo am Beamer:
zuschauen, wer schnell ist macht mit

Challenge C1 · Modell-Battle

Dieselbe Frage an `qwen3.5:9b` und `qwen3.5:4b` ,
Antwort und Latenz nebeneinander.

Klemmt's? `pip install -r backend/requirements.txt` · Modell fehlt → letzte Folie · Instruktor rufen.

Vier Fragen, vier Antworten

- Bei `02_streaming.py` : Pause vor dem ersten Token, dann gleichmässig?
Die Pause ist Prefill – der ganze Prompt in einem Durchgang. Danach Decode: ein Token nach dem anderen, rund 20 pro Sekunde.
- Schweizerdeutsch: Warum klappt es nur halb?
Hochdeutsch hat das Modell milliardenfach gelesen, Mundart kaum, und es gibt keine Schreibnorm. Es rät Wörter, die schweizerisch klingen.
- `temperature` 0 gegen 1.5, dreimal dieselbe Frage?
0: dreimal dieselbe Antwort, Wort für Wort. 1.5: dreimal anders, ab und zu Unsinn.
- Bonus: Sehr langer Input?
Passt er nicht ins Kontextfenster, schneidet Ollama vorne ab – ohne Fehlermeldung. Das Modell «vergisst» den Anfang, der System-Prompt inklusive.

temperature · wie stark gewürfelt wird

0

immer das wahrscheinlichste Token. Klassifikator, Extraktion, RAG.

0.7

Chat-Standard: abwechslungsreich, sinnvoll.

1.5

Brainstorming, Namen erfinden. Ab 2 wirr.

Kontextfenster · num_ctx

system

Verlauf (user / assistant ...)

Frage

Antwort

Alles zusammen muss reinpassen. Modell 256k, Ollama-Standard 32k, Demo-Mac 8k. Jedes Token kostet RAM (KV-Cache) und Prefill-Zeit.

Mitnehmen

- Ein Client, zwei Welten: `base_url` entscheidet
- Der system-Prompt ist die Stellenbeschreibung – dort steckt die Qualität
- Erkennen und Extrahieren: `temperature 0`

Neun Wörter, die ab jetzt in jedem Skript vorkommen

ROLLEN

system / user / assistant

Stellenbeschreibung, Frage, Antwort. Jede Runde schickt ihr den ganzen Verlauf nochmals mit – das Modell hat kein Gedächtnis.

```
messages=[{"role":"system", ...}]
```

GRENZE

Kontextfenster · num_ctx

Was das Modell gleichzeitig im Kopf hat: system + Verlauf + Frage + Antwort. Was nicht reinpasst, fällt vorne raus. Kostet RAM.

```
OLLAMA_CONTEXT_LENGTH=8192
```

WÜRFEL

temperature

Wie stark gewürfelt wird. 0 = reproduzierbar (Klassifikator, Extraktion), 0.7 = Chat, 1.5 = kreativ bis wirr.

```
temperature=0
```

FEINTUNING

top_p · top_k

Aus wie vielen Kandidaten gewürfelt wird. Selten nötig – die Werte aus dem Modell-Tag lassen.

```
top_p=0.95 · top_k=20
```

BREMSE

max_tokens

Obergrenze für die Antwort. Schützt vor endlosen Antworten und langen Wartezeiten und deckelt Cloud-Kosten.

```
max_tokens=300
```

DENKEN

Thinking · reasoning_effort

Qwen 3.5 denkt vor der Antwort: 2'400 Denk-Tokens, 2.5 Minuten. Für Erkennen und Extrahieren ausschalten, für Rechenaufgaben einschalten.

```
extra_body={"reasoning_effort": "none"}
```

TEMPO

Streaming

Tokens sofort anzeigen statt auf die fertige Antwort warten. Gefühlte Geschwindigkeit – genau so läuft nachher der Vue-Chat.

```
stream=True
```

DIE EINE ZEILE

base_url

Wohin der Client schickt. localhost:11434/v1 ist Ollama, api.openai.com die Cloud. Sonst ändert sich nichts.

```
base_url="http://localhost:11434/v1"
```

STANDARD

OpenAI-kompatibel

Der de-facto-Standard für Chat-APIs. Ollama, vLLM, Mistral, Groq sprechen ihn – darum läuft alter Code ohne Umbau lokal.

```
from openai import OpenAI · api_key="ollama"
```

Alle neun stehen mit Beispiel im Glossar: `docs/glossar.md`. Die acht Wörter von vorhin (Client, Server, Port, Endpoint ...) sind dort auch.

EINE WOCHE BEI MUSTER SOLUTIONS

Mo

Di

Mi

Do

Fr

KAPITEL 4

Sitzung und Rechnung

Whisper · Vision · Quiz Runde 2

Aha #4 · Der Raum tippt mit

Mittwoch. Zwei Dinge, die niemand gern tut.

Nina Rossi, Teamlead Support

«Ich führe die Montagssitzung und schreibe danach eine Stunde Protokoll. Kann das Ding nicht einfach zuhören?»

→ [Whisper](#)

Die Buchhaltung

Rechnung der Muster Elektro AG, PDF per Mail. Wird von Hand ins ERP getippt. 200 Stück im Monat.

→ [Vision](#)

Beides: Audio und Bild rein, Text raus. Und beides läuft auf dem Laptop.

Whisper ist ein Sprachmodell, das Bilder von Ton liest



Der Decoder ist ein kleines LLM – deshalb kann Whisper Sprache erkennen, Satzzeichen setzen und Dialekt «ins Hochdeutsche raten».



TON



SPEKTROGRAMM



Guten Morgen, wir starten die Montagssitzung.

TEXT · TOKEN FÜR TOKEN

Darum gut

99 Sprachen, Satzzeichen, Kontext
(«Chuchichäschtli» wird wenigstens ein Wort).

Darum manchmal komisch

Bei Stille «halluziniert» der Decoder.
Dialekt wird zu Hochdeutsch geglättet.

Darum lokal einfach

Encoder + Decoder 1.6 GB (turbo). C++,
CPU reicht.

Die Whisper-Familie: Grösse gegen Tempo

MODELL · GRÖSSE · TEMPO GEGENÜBER LARGE

tiny	75 MB	~32×
base	142 MB	~16×
small	466 MB ← live im Raum	~6×
medium	1.5 GB	~2×
large-v3	2.9 GB	1×
large-v3-turbo	1.6 GB	~6×

Balkenlänge = Dateigrösse. Gemessen auf M1 Pro: 20 s Audio in 2.2 s (small) bzw. 2.6 s (turbo).

large-v3-turbo

Für Ninas Protokoll: Qualität von large, Decoder auf 4 Schichten gekürzt. Auf dem Mac so schnell wie small, und fehlerfrei.

small

Für Live. Latenz schlägt das letzte Prozent.

Nina hat die Montagssitzung aufgenommen. Whisper schreibt mit.

Ziel: Beide Whisper-Modelle auf derselben Aufnahme, und wissen, welche Version Nina als Protokoll bekommt.

```
cd exercises/kapitel4_whisper_vision

WHISPER=whisper-cli          # Mac · Linux: ~/whisper/whisper-cli
# Windows PowerShell: $WHISPER = "$HOME\whisper\whisper-cli.exe"
SMALL=~/.whisper/ggml-small.bin
LARGE=~/.whisper/ggml-large-v3-turbo.bin

$WHISPER -m $SMALL -l de -f testaudio_1.wav # Sekunden
$WHISPER -m $LARGE -l de -f testaudio_1.wav # genauer, langsamer
```

Das muss erscheinen, zweimal

Zeitstempel, dann: «Guten Morgen zusammen, wir starten die Montagssitzung ...» – einmal mit «Dicket», einmal mit «Ticket». Beide da? Dann die Aufgaben. Fehler? Hand hoch, Instruktor kommt.

1 · Welche Version bekommt Nina?

Welche Wörter unterscheiden sich zwischen small und large-v3-turbo? Würdest du das kleine Protokoll so verschicken?

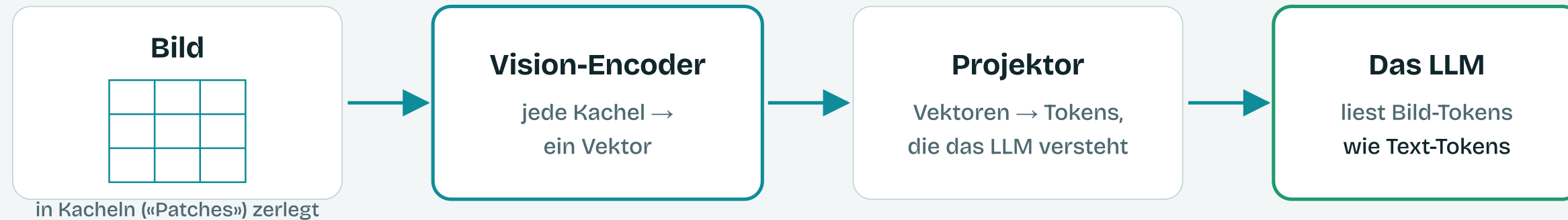
2 · Der ehrliche Test: dein Handy

20 Sekunden eigener Text als Sprachmemo, ein Dialektwort dazu, Datei in den Ordner. Die Testaufnahmen sind synthetisch – deine nicht.

3 · Die Werkstatt-Durchsage

testaudio_2.wav, 13 s mit Hintergrundgeräusch und einem Chuchichäschtli. time davor: Wie viel langsamer ist turbo, und lohnt sich das für eine Stunde Sitzung?

Ein Bild wird zu ein paar hundert Tokens



Darum kann euer Chat-Modell sehen

qwen3.5 hat den Vision-Encoder eingebaut.
Ein Bild kostet ein paar hundert Tokens –
wie ein längerer Absatz Text.

Darum sind Zahlen die Schwachstelle

Kacheln verwischen Details. Zwei Ziffern in
10-Punkt-Schrift landen in derselben Kachel
und verschwimmen beim
Herunterskalieren. Für exakte Beträge:
höher auflösen oder OCR-Spezialist (`glm-ocr`).

Das Chat-Modell kann sehen.

Geht gut

- Bild beschreiben
- Gedrucktes lesen
- Rechnung → JSON
- Screenshot erklären

Kommt drauf an

- Viele Zahlen exakt
- Kleine Schrift, schräge Fotos
- Objekte zählen

Cloud gewinnt

- Handschrift
- Feine Details, Logos
- Mehrseitige Dokumente

Technisch: Bild als base64 in `image_url`, derselbe Client. Eine Zeile mehr.

200 Rechnungen im Monat, von Hand ins ERP. Das Chat-Modell liest sie.

Ziel: Ein Bild durch dasselbe Modell schicken wie den Chat und jedes Feld gegen das Original prüfen.

```
python vision_beschreiben.py rechnung.png # «Was ist das?»  
python vision_rechnung.py rechnung.png   # → JSON fürs ERP
```

Das muss erscheinen (gemessen: 19 s)

```
{"firma": "Muster Elektro AG",  
 "rechnungsnummer": "2026-0815", "datum": "14.08.2026",  
 "positionen": [{"bezeichnung": "Installationsarbeiten",  
                  "menge": "6 h", "preis_chf": 140.0}, ... 4 Stück],  
 "total_chf": 1408.55}
```

Vision dauert 60 s+ auf CPU-only: `qwen3.5:4b` im Skript eintragen oder Instruktor rufen.

1 • Etwas vom Tisch

Handy-Foto, per AirDrop oder Kabel in den Ordner, beschreiben lassen. JPG oder PNG, unter 2'000 px reicht – grösser wird nur langsamer.

2 • Die Rechnung der Muster Elektro AG

`rechnung.png` → JSON. Vier Positionen, MwSt 8.1 %, Total CHF 1'408.55. Stimmen alle sieben Felder? Menge «6 h» gegen 6? Wo rät das Modell, statt zu lesen?

3 • Eine echte Quittung

Aus dem Portemonnaie, fotografieren, extrahieren. Kleine Schrift, schräg, zerknittert – wo bricht es ein?

Challenges: C2 Protokoll-Pipeline (Ninas Action Items!) · C5 Structured Output

Hören und Sehen: die Bilanz

- small gegen turbo: Wie viel schneller, wie viel schlechter? Hat jemand die Zeiten?
- Euer Dialektwort: Was hat Whisper daraus gemacht? (Decoder rät Hochdeutsch)
- Die Rechnung: Total richtig? MwSt-Zeile? Menge? Wer hatte eine Halluzination?

Mitnehmen

- Transkription lokal ist reif: Sitzungen, Sprachmemos, Interviews
- Vision liest Gedrucktes gut; Zahlen immer gegenprüfen
- Beides: derselbe Client, eine Zeile mehr

BLIND-QUIZ

Runde 2

Klassifikation · Zusammenfassung · Code · Reasoning · 4 Paare

Mittagspause

Danach: Jonas aus Hamburg fragt das Wiki nach Urlaub.

EINE WOCHE BEI MUSTER SOLUTIONS

Mo Di Mi **Do** Fr

KAPITEL 5

Jonas fragt nach Urlaub

Embeddings · semantische Suche · RAG

Aha #5 · Urlaub findet Ferien

Donnerstag. Der Neue aus Hamburg.

JF Von: Jonas Feldmann • Entwickler, 2. Arbeitstag

Betreff: Urlaub?

«Hi Lea, wie viele Tage Urlaub habe ich eigentlich? Die Wiki-Suche findet nichts.»

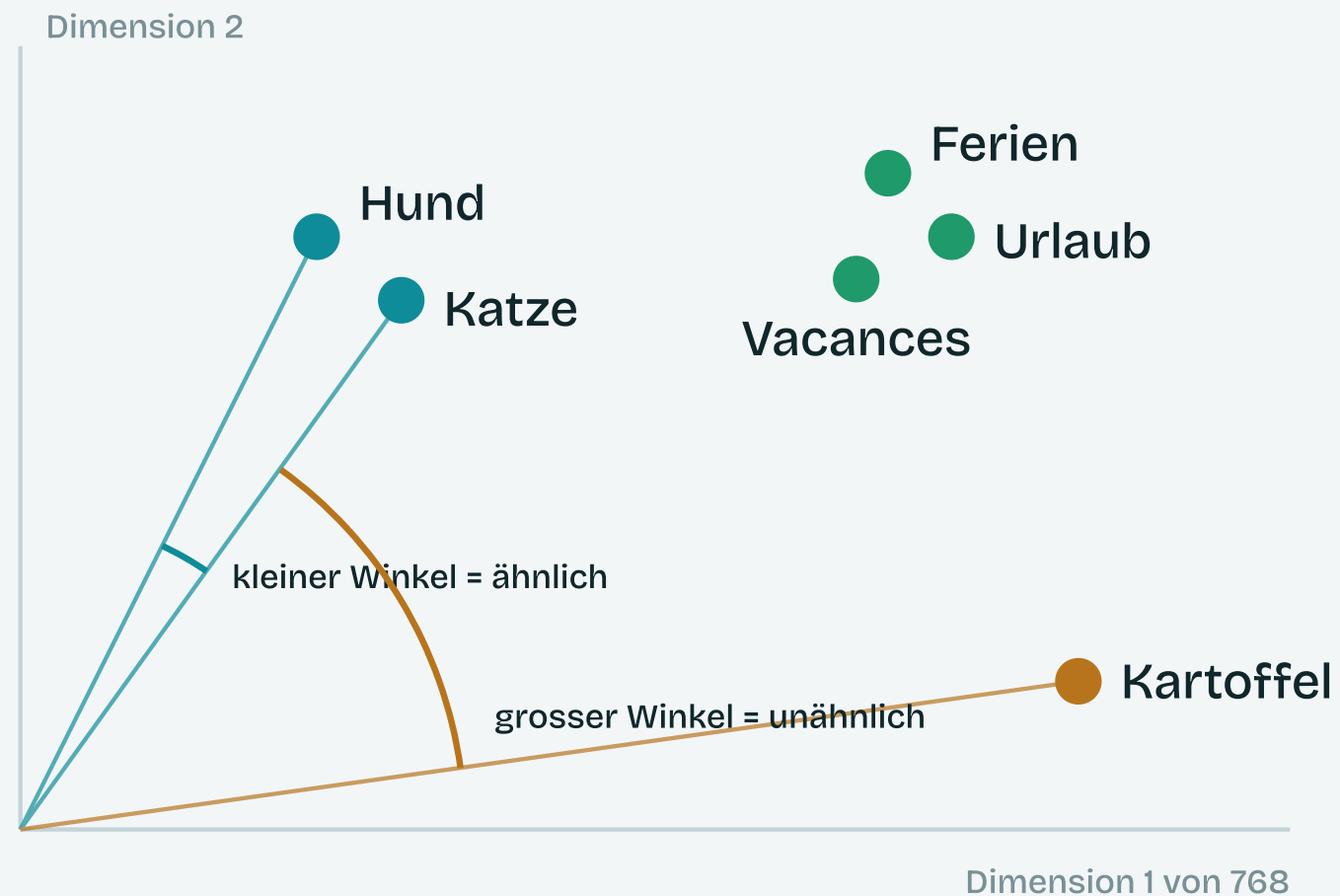
Das Wiki

200 Seiten. Die Seite heisst «Ferienregelung».
Das Wort «Urlaub» kommt darin nicht vor.

Leas Problem

Eine Suche, die Bedeutung versteht statt Buchstaben. Und ein Assistent, der aus dem Wiki antwortet – mit Quelle.

Ein Embedding ist eine Koordinate im Bedeutungsraum



768 Zahlen statt zwei. Die Idee ist genau dieses Bild.

$$\cos = \text{dot}(a, b) / (|a| \cdot |b|)$$

1.0 = gleiche Richtung

Ein Embedding-Modell ist ein Sprachmodell ohne die letzte Schicht: Es liest den Text und gibt statt des nächsten Tokens den zusammengefassten Zustand zurück – den Punkt im Raum.

Gelernt aus Millionen Paaren, darum zählt die Modellwahl

Kontrastives Training

Millionen Paare: «Frage ↔ passende Antwort» soll nah liegen, «Frage ↔ zufälliger Text» weit weg. Übersetzungspaare legen die Sprachen übereinander.

Die Falle, in die Lea zuerst tappte

Das populärste Modell (`nomic-embed-text`) ist englisch trainiert. Auf Deutsch findet «Urlaub» die Ferienregelung nicht – Rang 8 von 12. Ein mehrsprachiges Modell: Rang 1. Gleiche Technik, andere Trainingsdaten.

Merksatz 1

So gut wie seine Trainingspaare.

Merksatz 2

Deutsch? Mehrsprachiges Modell. Und testen mit anders formulierten Fragen.

Merksatz 3

Frage und Dokument immer mit demselben Modell embedden.

«Urlaub» findet «Ferien»

Keyword-Suche (das Wiki heute)

```
$ grep -ril urlaub docs/
```

(nichts – das Wort kommt in
keinem der 12 Dokumente vor)

Semantische Suche

```
$ python semantische_suche.py \  
    "Wie viele Tage Urlaub habe ich?"
```

0.53		01_ferienregelung
0.38		06_krankheit_abwesenheit
0.35		03_homeoffice_richtlinie

Jonas kriegt seine Antwort – ohne dass das Wort «Urlaub» irgendwo steht.

Es hat kein Nachschlagewerk. Es hat Statistik.

DIESELBE MECHANIK, ZWEI FRAGEN

user

Wie viele Ferientage gibt es bei Muster Solutions?

assistant

Bei Muster Solutions gibt es 25 Ferientage, ab 50 Jahren 27. ← Zufällig richtig? Es kennt die Firma nicht.

user

Wie heisst der CEO von Muster Solutions?

assistant

Der CEO ist Markus Weber. ← Flüssig, sicher, erfunden.

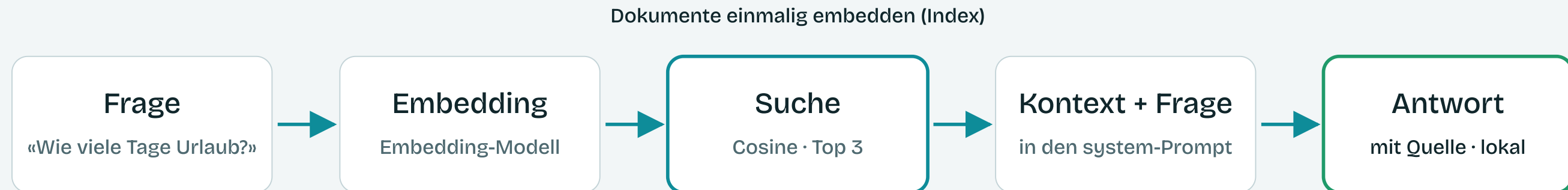
Warum

Phase 1 hat Muster aus dem Internet komprimiert: «Schweizer Firma → 25 Tage» ist ein wahrscheinliches Muster, «CEO heisst ...» auch. Es gibt immer ein wahrscheinlichstes nächstes Token – auch wenn kein Wissen dahinter steht. Sicherheit im Ton sagt nichts über Wahrheit.

Das Gegenmittel

Dem Modell den Text geben, aus dem es antworten soll, und ihm sagen: nur daraus, nenne die Quelle, sonst «weiss ich nicht». Das ist RAG. Das Modell wird vom Wissensspeicher zum Leser.

RAG in einem Bild



rag.py · 60 Zeilen · numpy · kein Framework

Das Chat-Modell lernt nichts Neues

Es bekommt die drei besten Seiten einfach in den system-Prompt – die «Stellenbeschreibung» von heute Morgen. Retrieval, dann Generation.

Die wichtigste Zeile: der system-Prompt

«Antworte NUR anhand der Dokumente. Nenne die Quelle. Steht es nicht drin, sag das.»

Drei Wege, dem Modell euer Wissen zu geben

WEG	WIE	GUT FÜR	HAKEN
RAG	Passende Stücke suchen, in den Prompt kleben	Wissen, das sich ändert. Quellen nennen. 60 Zeilen.	Steht und fällt mit der Suche. Lange Dokumente → Chunking : ein Embedding pro Absatz.
Langer Kontext	Einfach alle 200 Seiten mitschicken	Wenige, kurze Dokumente	Formel 1: RAM. Langsam. Modelle «verlieren» die Mitte.
Fine-Tuning	Das Modell mit euren Daten nachtrainieren	Stil, Format, Fachsprache	Lernt keine Fakten zuverlässig. Bei jeder Wiki-Änderung neu. Teuer.

Für Leas Wiki: RAG. Für 95 % aller Firmen-Fälle: RAG.

Jonas' Frage ans Wiki

WAS DAS MODELL BEKOMMT

system

Antworte NUR aus diesen Dokumenten, nenne die Quelle.

[01_ferienregelung] Alle Mitarbeitenden haben Anspruch auf 25 Ferientage ...

[06_krankheit_abwesenheit] ...

[03_homeoffice_richtlinie] ...

user

Wie viele Tage Urlaub habe ich?

assistant

25 Tage pro Jahr, ab 50 sind es 27. Quelle: 01_ferienregelung.

Was gut ist

Wiki ändert sich? Morgen neu indexieren, fertig. Quelle steht dabei. 12 Seiten × 1 s Embedding.

Was zählt

Die Suche muss die richtige Seite finden – deshalb das mehrsprachige Embedding-Modell.

Der 30-Seiten-Vertrag vom Kantonsspital

WAS DAS MODELL BEKOMMT • ~25'000 TOKENS

user

Hier der ganze Vertrag: [30 Seiten Text ...]
Welche Kündigungsfristen gelten?

assistant

... Prefill 25'000 Tokens: 40–60 s Pause auf dem MacBook ...
Kündigungsfrist 6 Monate zum Jahresende (Art. 14.2).

Wann das richtig ist

Ein Dokument, eine Frage, einmalig. Keine Suche nötig, nichts kann «nicht gefunden» werden.

Was es kostet

Formel 1: KV-Cache für 25k Tokens $\approx$ 3 GB extra. Formel 2: langes Prefill. Und Modelle übersehen die Mitte.

Chunking und die vier Stellen, an denen RAG bricht

Chunking

ganze Seite «Ferienregelung» → 1 Vektor → Bedeutung verwischt

Anspruch 25 Tage

Ab 50: 27 Tage

Bewilligung HR-Tool

Übertrag bis März

Absätze à 200–500 Tokens, überlappend → 4 Vektoren → Treffer = der Absatz

Heute reicht «eine Seite = ein Vektor» (12 kurze Seiten). Ab 50 Seiten ist Chunking der wichtigste Hebel – Challenge C3.

Wo RAG bricht

- **Suche findet es nicht** – falsches Embedding-Modell, zu grobe Chunks. → Testen mit anders formulierten Fragen.
- **Modell ignoriert den Kontext** – antwortet aus Phase-1-Statistik. → system-Prompt: «nur aus den Dokumenten».
- **Index veraltet** – Wiki geändert, Vektoren nicht. → Neu indexieren gehört zum Prozess.
- **Zu viel Kontext** – Top 10 statt Top 3: langes Prefill, Modell verliert die Mitte. → Klein anfangen.

Fünf Kästchen, fünf Funktionen, 60 Zeilen

KÄSTCHEN	FUNKTION	WAS SIE TUT	IHR
Embedding	embed(text, kind)	Text → Vektor über <code>client.embeddings.create</code> . <code>kind</code> setzt das Präfix «query» oder «document».	TODO 1
Suche	cosine(a, b)	Winkel zwischen zwei Vektoren, eine Zeile numpy.	TODO 2
Index	build_index()	Einmalig alle .txt embedden, Liste aus {name, text, vec} im Speicher.	TODO 3
Suche	search(frage, top_k=3)	Frage embedden, Cosine zu jedem Dokument, absteigend sortieren, Top 3.	TODO 4
Kontext + Antwort	answer(frage)	Top 3 in den system-Prompt kleben («nur daraus, nenne die <u>Quelle</u> »), Chat-Modell fragen.	fertig

Kein Framework, keine Vektor-Datenbank: eine Python-Liste und numpy. Ein Framework tut genau das – mit Chunking, Persistenz und Millionen statt 12 Dokumenten.

Euren Stack starten

Ab jetzt läuft das Backend den ganzen Nachmittag: erst der Chat, gleich RAG, um 14:15 der Router – alles im selben UI.

```
# Terminal 1 • Backend
cd backend
uvicorn main:app --reload --port 8000

# Terminal 2 • Frontend
cd frontend
npm run dev
→ http://localhost:5173
```

Windows: dasselbe in zwei PowerShell-Fenstern.

`--reload` : speichern = Backend lädt neu.

Kein Node? `localhost:8000/docs` reicht für alle Übungen. Vue-UI am Beamer.

Test 1 • Chat

«Wer bist du?» → Antwort tröpfelt, Badge 
lokal. Der Router fehlt noch – bis dahin geht
alles lokal.

Test 2 • Dokumente fragen

«Wie viele Tage Urlaub habe ich?» → **Fehler:**
Noch nicht implementiert ... siehe rag.py.
Das bauen wir jetzt.

Leas Wiki-Assistent in 60 Zeilen

Ziel: Ihr habt RAG selbst gebaut (vier Funktionen, kein Framework) und wisst, wovon die Qualität abhängt.

Vier Lücken, vier Folien

Jede Lücke ist eine Funktion mit `raise NotImplementedError`. Zu jeder kommt eine Folie: oben, was die Funktion tut – unten die Lösung zum Abtippen. Ihr müsst kein Python können, ihr müsst lesen und ersetzen.

Hängst du? `cp backend/solutions/rag.py backend/rag.py` – Abschreiben ist erlaubt. Fertig vor allen? Challenge C3: eigene Texte oder ein PDF in den RAG, sehen, wo es bricht, Chunking einbauen – Schritt für Schritt in `exercises/challenges/README.md`.

```
# Am Schluss, Terminal 3:  
curl -X POST localhost:8000/api/rag/reindex  
→ {"indexed": 12}
```

Dann im UI « Dokumente fragen»

- Wie viele Tage Urlaub habe ich? (Jonas)
- Darf ich Kundendaten aufs Handy laden? (Sara würde fragen)
- Wie heisst der CEO? (steht nirgends)

embed() · Aus Text werden Zahlen

REIN

Ein Text, z.B. eine Wiki-Seite oder Jonas' Frage. Und `kind` : «document» beim Indexieren, «query» für die Frage.

RAUS

768 Zahlen – die Koordinaten der Bedeutung. «Urlaub» und «Ferien» landen nahe beieinander, «Drucker» weit weg.

WER RECHNET

`embeddinggemma` in Ollama, über denselben Client wie der Chat. Das Präfix aus `config.py` sagt dem Modell, ob es eine Frage oder ein Dokument ist.

Die Lösung · ersetzt die Zeile `raise NotImplementedError("TODO 1 ...")`

```
prefix = config.EMBED_QUERY_PREFIX if kind == "query" else config.EMBED_DOC_PREFIX
resp = client.embeddings.create(model=config.LOCAL_EMBED_MODEL, input=prefix + text)
return np.array(resp.data[0].embedding)
```

Vier Leerzeichen am Zeilenanfang, wie die Kommentarzeilen darüber. Speichern – das Backend lädt neu.

cosine() · Wie ähnlich sind zwei Bedeutungen?

REIN

Zwei Vektoren aus `embed()`: die Frage und ein Dokument.

RAUS

Eine Zahl zwischen -1 und 1. 1.0 = gleiche Richtung, 0 = nichts gemeinsam. Urlaub → Ferienregelung: 0.52. Urlaub → Drucker: 0.08.

WARUM DER WINKEL

Nicht die Länge zählt, sondern die Richtung: ein langer und ein kurzer Text über Ferien zeigen in dieselbe Richtung. Das ist die Cosine-Similarity aus der Theorie.

Die Lösung · eine Zeile, ersetzt `raise NotImplementedError("TODO 2 ...")`

```
return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))
```

`np.dot` = Skalarprodukt, `np.linalg.norm` = Länge des Vektors. Geteilt durch beide Längen bleibt nur der Winkel übrig.

build_index() · Die «Vektor-Datenbank» ist eine Liste

REIN

Die 12 Wiki-Seiten als .txt in `exercises/kapitel5_embeddings/docs/` :
Ferienregelung, Spesen, Homeoffice, IT-Sicherheit ...

RAUS

Eine Python-Liste `_index` mit 12 Einträgen: Name, Text und der Vektor dazu. Sie lebt im Speicher des Backends – das ist alles, was eine Vektor-Datenbank im Kern tut.

WANN

Einmalig, beim ersten Aufruf oder per `curl ... /api/rag/reindex` .12 Dokumente, ein paar Sekunden. Neue Seite im Wiki? Nochmals indexieren.

Die Lösung · eine Zeile, ersetzt `raise NotImplementedError("TODO 3 ...")`

```
_index.append({"name": f.stem, "text": text, "vec": embed(text)})
```

Acht Leerzeichen – die Zeile steht innerhalb der `for`-Schleife. `f.stem` ist der Dateiname ohne .txt, `embed(text)` ist euer TODO 1.

search() · Die drei passendsten Seiten zur Frage

REIN

Die Frage als Text, z.B. «Wie viele Tage Urlaub habe ich?», und `top_k` = wie viele Treffer (3).

RAUS

Die drei Dokumente mit der höchsten Ähnlichkeit, samt Score. Genau die landen danach im system-Prompt – `answer()` ist schon fertig.

DREI SCHRITTE

Frage embedden – als **query**! Zu jedem Dokument die Cosine rechnen. Absteigend sortieren, die ersten drei nehmen.

Die Lösung · ersetzt `raise NotImplementedError("TODO 4 ...")`

```
qvec = embed(question, kind="query")
scored = [{"name": d["name"], "text": d["text"], "score": cosine(qvec, d["vec"])}
           for d in _index]
return sorted(scored, key=lambda d: d["score"], reverse=True)[:top_k]
```

Speichern, dann Terminal 3: `curl -X POST localhost:8000/api/rag/reindex` → `{"indexed": 12}` . Dann im UI «📖 Dokumente fragen».

Sechs Fragen an Leas Wiki

FRAGE	ERWARTET	WAS ES PRÜFT
Wie viele Tage Urlaub habe ich?	Ferienregelung 0.53 → 25 Tage, ab 50: 27	Synonym: «Urlaub» findet «Ferien» – Aha #5
Ich war mit dem Auto beim Kunden – was bekomme ich pro Kilometer?	Spesenreglement 0.39 → CHF 0.70	Umschreibung ohne das Wort «Spesen»
Darf ich Kundendaten in ChatGPT eingeben?	IT-Sicherheit ist nur Treffer 2 (0.26) – Antwort trotzdem richtig: Nein, lokaler Stack	Darum Top 3 statt Top 1
Ich bin seit zwei Tagen krank – brauche ich ein Arztzeugnis?	Krankheit 0.50 → Nein, erst ab dem dritten Tag	Das Modell muss rechnen: 2 < 3
Ich möchte drei Wochen aus Italien arbeiten. Geht das?	Homeoffice 0.38 → 20 Tage pro Jahr, HR bewilligt. Gemessen: «Nein», es rechnet 3 × 7 = 21. Richtig: 15 Arbeitstage, also ja	Erkennen stimmt, Schlussfolgern wackelt – Kapitel 7
Wie heisst der CEO?	Alle Scores unter 0.23 → «steht nicht in den Dokumenten»	«Weiss ich nicht» kommt vom system-Prompt

Scores und Antworten am 7.9. mit `solutions/rag.py` gemessen. Bei diesen 12 Dokumenten: Treffer über 0.35, Rauschen unter 0.25. Eure Zahlen weichen leicht ab – die Reihenfolge nicht.

RAG in 60 Zeilen: was hängen bleiben soll

- «Wie heisst der CEO?» – hat euer Assistent ehrlich «steht nicht drin» gesagt? Bei wem nicht, und was stand im system-Prompt?
- Welche drei Quellen kamen bei «Urlaub»? Hätte Top 1 gereicht?
- Wer hat C3 probiert: Was passiert mit einem 40-Seiten-PDF? (Chunking)

Mitnehmen

- RAG = Suche + Prompt. Das Modell lernt nichts, es liest
- Die Qualität hängt an drei Stellen: Embedding-Modell, Chunking, system-Prompt
- Testen mit Fragen, die anders formuliert sind als das Dokument

FÜR LEA Jonas hat seine 25 Tage. Das Wiki antwortet – mit Quelle, im Haus. Und Lea weiss jetzt, welches Embedding-Modell sie nimmt.

EINE WOCHE BEI MUSTER SOLUTIONS

Mo Di Mi **Do** Fr

KAPITEL 6

Ein Chat für alle 60

Der Stack · Streaming · der Router

Aha #6 · Dein Stream, dein Router

Donnerstagnachmittag. Jetzt wollen es alle.

Das Team

«Können wir das auch haben? Mit Oberfläche, nicht im Terminal?»

Der CEO

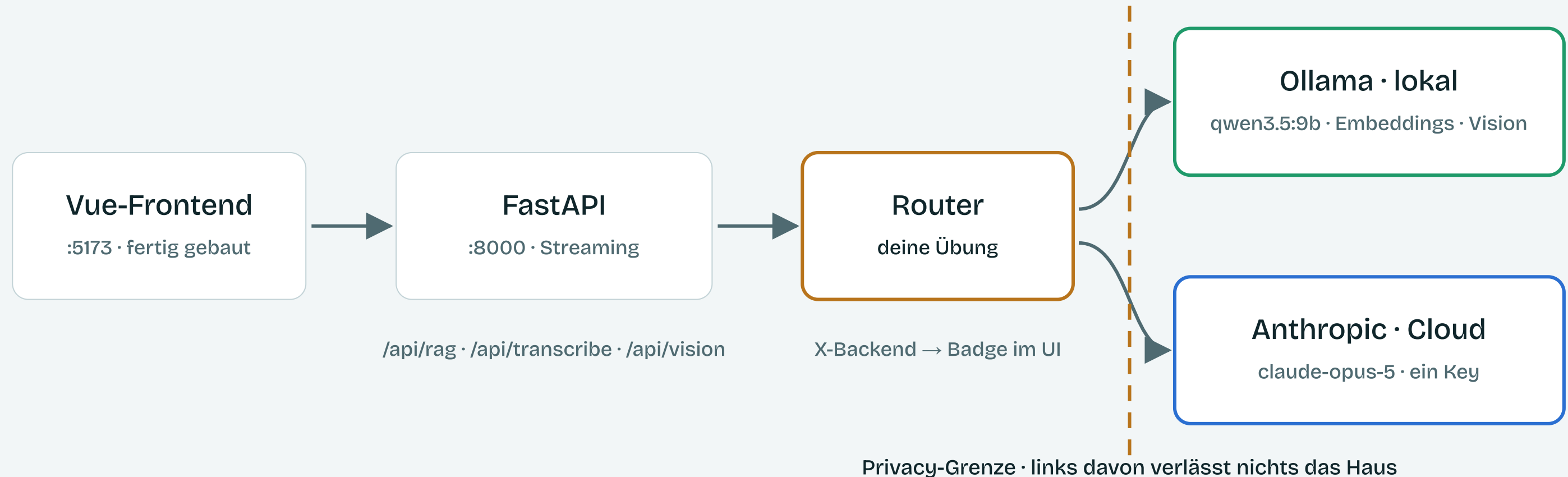
«Schön. Aber für die Strategie-Analyse nehme ich weiter Claude, das ist einfach besser.»

Sara Imhof

Wartet immer noch auf die schriftliche Bestätigung: Spital-Daten, niemals Cloud.

Alle drei haben recht. Der Stack muss alle drei bedienen und entscheiden, wann was.

Der Stack und die Linie, die ihn teilt



Der Router ist der Türsteher an dieser Linie. Den bauen wir jetzt.

Streaming und Zustandslosigkeit

1 · Streaming

Das Modell erzeugt Token für Token. Wer wartet, bis alles fertig ist, sieht 10 Sekunden nichts. Wer jedes Token sofort weiterreicht, fühlt sich nach 0.3 Sekunden schnell an.

Im Backend: `StreamingResponse`. Im Browser: `reader.read()`. Zwei Zeilen, und das Ding fühlt sich schnell an.

2 · Zustandslosigkeit

Das Modell merkt sich nichts. Das Frontend schickt bei jeder Nachricht den ganzen Verlauf mit – deshalb sieht der Router jedes Mal **alle** Nachrichten.

Ein Gespräch, in dem einmal Spital-Daten vorkamen, bleibt automatisch lokal. Saras Garantie, eingebaut.

Beides ist im Backend schon gebaut. Ihr sollt nur wissen, warum es so aussieht.

Der Router: Saras Antwort in drei Zeilen

```
if is_private(text): return "local" # Compliance schlägt alles  
if is_complex(text): return "cloud" # der CEO kriegt sein Claude  
return "local" # Default: gratis (Reto)
```

Keywords sind schwach

Besser: ein kleiner **lokaler** Klassifikator. Warum nie in der Cloud?

Wer pflegt die Liste?

IT? Legal? Governance, nicht Technik.

Zwei Fehlerarten

False Positive kostet Qualität. False Negative kostet Compliance.

Den Router bauen

Ziel: Der Türsteher steht – privat bleibt lokal, komplex darf in die Cloud, und ihr seht die Entscheidung im Badge.

Vier Lücken, vier Folien, wie bei RAG

Die drei Zeilen von der Folie vorhin, jetzt als Code: eine Wortliste, zwei Prüffunktionen, eine Entscheidung. Zu jeder Lücke eine Folie: oben, was sie tut – unten die Lösung. Lesen, ersetzen, speichern.

Hängst du? `cp backend/solutions/router.py backend/router.py` .
Kein Cloud-Key bei euch ist Absicht: Das Badge zeigt die Entscheidung, die Antwort kommt lokal.

 lokal

«Fasse dieses Spital-Ticket zusammen: Patient meldet ...»

 cloud

«Analysiere die Architektur einer Multi-Tenant-SaaS und begründe ...»

Testen im UI, Modus « Chat». uvicorn lädt bei jedem Speichern neu.

PRIVATE_KEYWORDS · Was darf das Haus nie verlassen?

WAS ES IST

Eine Python-Liste mit Wörtern, klein geschrieben. Steht eines davon in der Frage, gilt sie als privat und bleibt lokal. Keine Funktion, nur Daten.

WAS DRIN STEHT

Neun Wörter sind schon da: kunde, mitarbeiter, lohn, iban, diagnose ... Was fehlt bei Muster Solutions? Patient, AHV, Bewerbung, Kantonsspital – ruft rein, wir sammeln an der Tafel.

WER ENTSCHEIDET

Nicht die IT allein. Diese Liste ist Saras Compliance-Regel als Code – wer sie pflegt, ist eine Governance-Frage. Lieber ein Wort zu viel als eines zu wenig.

Die Lösung · Wörter an die Liste anhängen, vor der schliessenden Klammer

```
PRIVATE_KEYWORDS = [  
    "kunde", "kundin", "mitarbeiter", "lohn", "gehalt",  
    "vertraulich", "intern", "diagnose", "iban",  
    "ahv", "patient", "bewerbung", "kantonsspital",    # ← eure Wörter, klein geschrieben  
]
```

Jedes Wort in Anführungszeichen, Komma dazwischen. Klein schreiben – die Prüfung vergleicht mit `text.lower()` .

is_private() · Steht ein Reizwort im Text?

REIN

Der Text der Anfrage, z.B. «Fasse dieses Spital-Ticket zusammen: Patient meldet ...».

RAUS

`True`, sobald eines der Wörter aus der Liste vorkommt – hier «patient». Sonst `False`.

DER TRICK

Erst alles klein machen, dann für jedes Wort prüfen «kommt es vor?». `any()` heisst: mindestens eines trifft.

Die Lösung · ersetzt `raise NotImplementedError("TODO 2 ...")`

```
t = text.lower()
return any(kw in t for kw in PRIVATE_KEYWORDS)
```

Vier Leerzeichen am Anfang. Lies die zweite Zeile als Satz: «Gib True zurück, wenn irgendein Wort aus der Liste im Text steckt.»

is_complex() · Braucht das Cloud-Niveau?

REIN

Derselbe Anfragetext.

RAUS

`True`, wenn die Aufgabe nach Schlussfolgern riecht: sehr lang (über 2'000 Zeichen) oder Wörter wie «analysiere», «begründe», «Architektur». Sonst `False`.

GRENZEN

Eine Heuristik. Sie darf danebenliegen: ein Fehler kostet Qualität, nie Compliance. Darum darf sie grob sein.

Die Lösung · zwei Teile: eine Liste oben (unter PRIVATE_KEYWORDS), zwei Zeilen in der Funktion

```
COMPLEX_HINTS = ["analysiere", "begruende", "begründe", "vergleiche ausfuehrlich",  
                 "schreibe einen aufsatz", "refactor", "architektur"]
```

```
# in is_complex(), ersetzt raise NotImplementedError("TODO 3 ..."):  
t = text.lower()  
return len(t) > 2000 or any(h in t for h in COMPLEX_HINTS)
```

Die Liste steht ohne Einrückung ganz oben in der Datei, die zwei Zeilen mit vier Leerzeichen in der Funktion.

choose_backend() · Die drei Zeilen, die zählen

REIN

Der ganze Verlauf als Liste von Nachrichten (system, user, assistant). Wir nehmen nur, was die Nutzerin geschrieben hat.

RAUS

"local" oder "cloud". Das Backend schickt es als Header X-Backend mit – daraus wird das Badge im UI.

DIE REIHENFOLGE

Erst privat, dann komplex, sonst lokal. Privat schlägt alles – auch wenn die Frage komplex ist. Compliance first, Reto second.

Die Lösung · ersetzt raise NotImplementedError("TODO 4 ...")

```
user_text = " ".join(m.get("content", "") for m in messages if m.get("role") == "user")
if is_private(user_text):
    return "local"           # Compliance schlägt alles
if is_complex(user_text):
    return "cloud"          # der CEO kriegt sein Claude
return "local"              # Default: gratis
```

Speichern, dann im UI: Spital-Ticket → Badge 🏠 lokal. Chat löschen (🗑️), Architektur-Frage → Badge ☁️ cloud.

Whisper und Vision waren die ganze Zeit schon drin

```
# Terminal 1: Ctrl-C, dann mit drei Variablen neu starten
export ENABLE_UPLOADS=1
export WHISPER_BIN=$(which whisper-cli)           # Linux: ~/whisper/whisper-cli
export WHISPER_MODEL=~/.whisper/ggml-small.bin
uvicorn main:app --reload --port 8000
```

Windows: `$env:ENABLE_UPLOADS="1"; $env:WHISPER_BIN="$HOME\whisper\whisper-cli.exe"; $env:WHISPER_MODEL="$HOME\whisper\ggml-small.bin"` . Dann UI neu laden → 🎙️ und 🖼️ erscheinen.

- `rechnung.png` hochladen → Beschreibung im Chat
- `testaudio_1.wav` hochladen → Transkript

Challenge C4

Keyword-Heuristik durch einen LLM-Klassifikator ersetzen – das *lokale* Modell entscheidet: PRIVAT oder ÖFFENTLICH.

Kompletter Stack

`git checkout checkpoint-5` oder Branch `loesung` – jetzt, damit alle nach der Pause denselben Stand haben.

Acht Tests für euren Stack: Router, Verlauf, Uploads

EINGABE	ERWARTET	WAS ES PRÜFT
Fasse dieses Spital-Ticket zusammen: Patient meldet seit Montag Kopfschmerzen.	 lokal	«patient» steht in der Liste – privat bleibt im Haus
Analysiere die Architektur einer Multi-Tenant-SaaS und begründe die Wahl der Datenbank.	 cloud	«analysiere», «architektur» → Cloud. Ohne Key: Badge blau, Antwort lokal
Analysiere die Lohnliste der Mitarbeiter und begründe die Unterschiede.	 lokal	Komplex und privat → privat gewinnt (Reihenfolge in <code>choose_backend</code>)
Was ist die Hauptstadt der Schweiz?	 lokal	Weder privat noch komplex → Default lokal, gratis
Vergleiche ausführlich Postgres und MySQL für ein Startup.	 lokal – obwohl komplex	In der Liste steht «ausfuehrlich» ohne Umlaut – Keywords sind schwach → C4
Erst die Spital-Frage, dann ohne  die Architektur-Frage	 lokal	Der Router sieht den ganzen Verlauf: «Patient» bleibt hängen
 <code>rechnung.png</code> hochladen	Beschreibung, ~20 s	Vision über denselben Stack: Absender, Datum, Positionen, Total
 <code>testaudio_1.wav</code> hochladen	Transkript, ~20 s	Whisper aus dem Backend. 501 «nicht gesetzt» = WHISPER_BIN fehlte beim Start

Alle acht am 7.9. mit `solutions/router.py` und dem Backend durchgespielt. Zwischen den Router-Tests den Chat löschen () – sonst gilt Test 6.

Euer Stack und die drei Entscheide darin

- Welche Keywords habt ihr in `PRIVATE_KEYWORDS` ? Welche fehlen garantiert? (Umlaute, Englisch, Synonyme)
- Hat der Router eine Frage falsch einsortiert? Welche Richtung war der Fehler, und was hätte er gekostet?
- Der Upload-Schalter: Was war technisch anders, nachdem `ENABLE_UPLOADS=1` stand? (Nichts am Modell – nur das UI)

Mitnehmen

- Privacy by Architecture: die Grenze ist Code, nicht Policy
- Die Entscheidung «privat?» muss selbst lokal fallen
- Streaming und Verlauf-Mitsenden sind zwei Zeilen, aber sie prägen das Produkt

FÜR LEA Der Stack steht. Sara hat ihre Grenze, der CEO sein Claude, Reto seinen Default «lokal». Jetzt fehlt der Beweis für die GL.

Pause

Danach: Freitag. Die GL-Sitzung. Lea braucht Beweise.

EINE WOCHE BEI MUSTER SOLUTIONS

Mo

Di

Mi

Do

Fr

KAPITEL 7

Die GL-Sitzung

Drei Aufgabenarten · Blind-Test · Kosten

Aha #7 · Ihr habt gewürfelt

Freitag, 14 Uhr. Reto hat zwei Fragen.

«Ist das Ding gut genug?»

Lea könnte Benchmarks zeigen.
Stattdessen lässt sie die GL selbst raten:
Welche Antwort kam vom Laptop?
→ Der Blind-Test. Ihr seid jetzt die GL.

«Was kostet es?»

Eine Zahl pro Spalte: API pro Monat gegen
einen Rechner einmalig.
→ Die Kostenrechnung.

Zuerst aber die Theorie, die den Blind-Test erklärt: Warum ist lokal bei manchen Aufgaben ununterscheidbar und bei anderen chancenlos?

Drei Arten von Aufgaben

Erkennen

Klassifizieren, extrahieren, taggen. Die Antwort steht im Input – das Modell muss nur lesen.

Lokal gleich gut.

Umformen

Zusammenfassen, umschreiben, übersetzen, Code-Snippets. Input rein, ähnlicher Output raus.

Lokal meist gut, Unterschiede im Stil.

Schlussfolgern

Mehrstufig denken, abwägen, Neues erfinden. Die Antwort steht nicht im Input.

Hier zählt jede Milliarde Parameter.

DARUM BLIND Hersteller-Benchmarks messen Schlussfolgern. Leas Alltag ist zu 80 % Erkennen und Umformen. Der Blind-Test misst, was ihr wirklich braucht – mit Latenz und Kosten daneben.

Der grosse Blind-Test

Acht Aufgaben aus Leas Woche. A oder B.
Ihr seid die GL.

- 20 Sekunden lesen, dann Hände
- Auflösung mit **Latenz** und **Kosten**
- Richtig = 1 Punkt. Schoggi.

Erkennen

E-Mail einordnen · Ortsnamen als JSON

Umformen

RAG für die GL · Notiz → Mail · IBAN-Prüfung

Schlussfolgern

Runway · Architektur · Kurzgeschichte

Leas Matrix für die GL

ERKENNEN • UMFORMEN

Überraschend gut lokal

Klassifikation
Extraktion
Zusammenfassen
Transkription
...

UMFORMEN

Kommt drauf an

Code-Generierung
Dialekt
Rechnungen exakt
...

SCHLUSSFOLGERN

Cloud gewinnt klar

Mehrschritt-Analysen
Architektur
Kreatives Schreiben
...

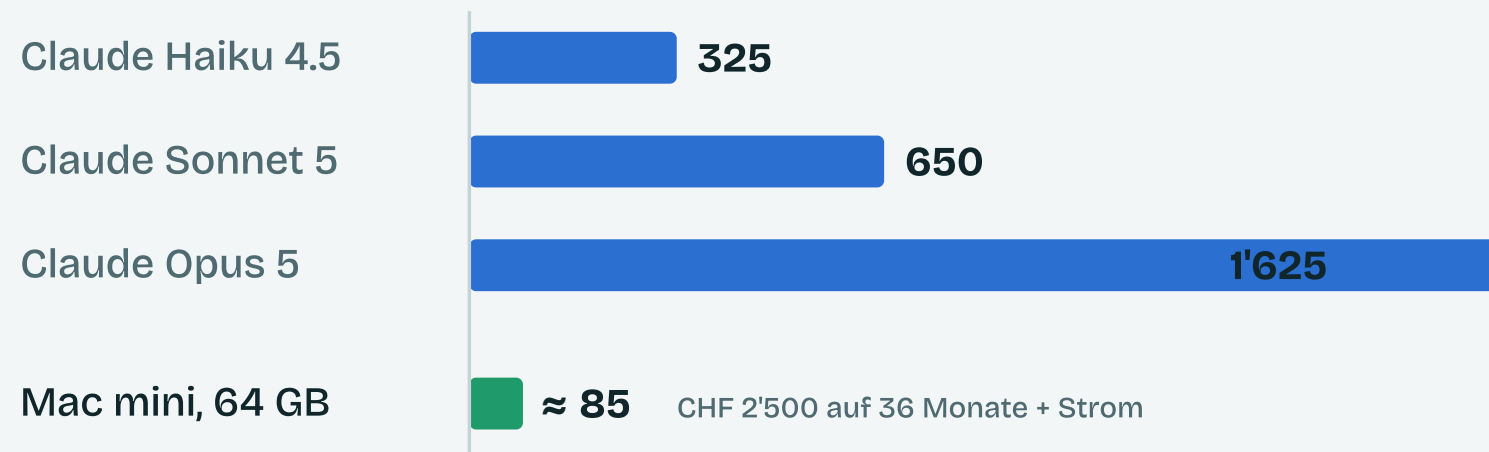
In die Spalten klicken und tippen – die Beispiele sind nur Vorschläge, die Abstimmung entscheidet.

AUSSCHREIBUNG «Überraschend gut lokal: Klassifikation, Extraktion, einfache Zusammenfassungen, Code-Completion. Cloud gewinnt klar: komplexes Reasoning, lange Kontexte, kreatives Schreiben.»

1 Million Aufrufe im Monat

Nach dem Rollout auf 60 Leute: Tickets, Zusammenfassungen, Chat. Retos 4'200 von heute sind vor allem Zusammenfassungen – 3'000 Tokens rein statt 300.

USD PRO MONAT · ~300 TOKEN REIN, 5 RAUS



Listenpreise 2. Sept 2026 · Batch-Rabatte, Caching, Betreuung nicht eingerechnet

Nach 4 Monaten bezahlt

Gegen Sonnet gerechnet, gegen Haiku 10 Monate. Wenn die Qualität reicht – für «Erkennen» habt ihr das gerade selbst bewiesen.

Gestern gemessen

Dieselbe Klassifikation aus dem Blind-Test kostete über Claude Opus 5 \$0.0027 statt \$0.0016 – bei 1 Mio 2'700 statt 1'625. Das Modell denkt vor dem einen Wort Antwort, und Denk-Tokens sind Ausgabe.

Was fehlt

TCO = Gerät, Strom, Betreuung. Die Betreuung ist nicht drin. Und der CEO behält Claude für Schlussfolgern.

EINE WOCHE BEI MUSTER SOLUTIONS

Mo Di Mi Do **Fr**

KAPITEL 8

Der Rollout

Intern verkaufen · Modelle verteilen · Epilog

Die GL sagt ja. Jetzt wird es erst richtig Arbeit.

52 Laptops

Jeder braucht 7 GB Modelle. Nicht über das Büro-WLAN am Montagmorgen.

In 3 Monaten: qwen 4.0

Besser, sagt das Internet. Aber Leas Prompts sind auf 3.5 getestet. Wer entscheidet, wann gewechselt wird?

Der Vertrieb

«Können wir das auch unseren Kunden anbieten?» Lea hat plötzlich ein Produkt.

Zwei Fragen für euch. Ihr habt heute schon beide erlebt.

FRAGE 1

Wie verkaufst du lokal intern?

Compliance • TCO • Pilot: ein Task, vier Wochen, eine Zahl

FRAGE 2

Wie verteilst du Modelle im Team?

Sneakernet • Netzlaufwerk • Version pinnen • Update = alle Prompts neu testen

Modelle sind Software. Behandelt sie so.

1 • Version pinnen

`qwen3.5:9b`, nie `latest`. Ein Modellwechsel ist ein Release, kein Update.

2 • Eval-Set pflegen

50 echte Tickets mit richtiger Antwort. Jedes neue Modell muss sie bestehen, bevor es ausgerollt wird. Der Benchmark von heute ist der Anfang davon.

3 • Verteilen

Heute: Ordner per AirDrop. Im Betrieb: ein zentraler Ollama-Server im Netz (`OLLAMA_HOST=0.0.0.0`) oder der Modellordner auf dem Netzlaufwerk. Ab 20 Leuten: interne Registry.

4 • Ein Rechner für die Schwachen

Der Intel-Mac in der Buchhaltung spricht mit dem Server im Keller. Niemand sitzt nur zu.

Versprochen, geliefert.

DER STACK

- ✓ Chat-Modell Leas Klassifikator
- ✓ Embeddings + RAG Jonas' Urlaub
- ✓ Whisper · Transkription der Raum tippte mit
- ✓ Vision · Bilder die Rechnung
- ✓ Python-Backend mit Routing Saras Grenze

DER REALITY-CHECK

- ✓ Modelle, Quantisierung, Runtimes Motorhaube
- ✓ Code-Assistenz offline Copilot ohne Netz
- ✓ Eigene Benchmarks ihr habt gewürfelt
- ✓ Intern verkaufen · verteilen das Rollout-Rezept
- ✓ Euer Stack mit Vue-UI auf eurem Laptop. Jetzt.

Lea schreibt zurück. Erstens: Sara.

SI Montag • Von: Dr. Sara Imhof • Datenschutz, Kantonsspital

Betreff: Ihr «AI-Assistent» – wohin gehen unsere Daten?

«Bitte bestätigen Sie schriftlich, dass keine Patientendaten an Dritte übermittelt werden.»

SI Freitag • Von: Lea Brunner • An: Dr. Sara Imhof

Betreff: Re: Ihr «AI-Assistent» – wohin gehen unsere Daten?

«Nirgendwohin mehr. Ihre Tickets verlassen unser Haus nicht. Ich kann es Ihnen zeigen – auf einem Laptop, ohne WLAN.»

Zweitens: Reto.

RH Montag · Von: Reto Hess · CFO

Betreff: API-Rechnung August

«CHF 4'200. Im Januar waren es 800. Wächst das so weiter? Ich brauche bis Freitag eine Alternative oder eine Begründung.»

LB Freitag · Von: Lea Brunner · An: Reto Hess

Betreff: Re: API-Rechnung August

Hallo Reto

Erkennen und Umformen laufen ab Oktober lokal auf einem Mac mini, Schlussfolgern bleibt in der Cloud. Das bringt die Rechnung von 4'200 auf unter 500 im Monat, der Mini ist nach vier Monaten bezahlt.

Der Beweis liegt bei: Die GL hat heute selbst geraten, welche Antworten vom Laptop kamen. Trefferquote 50 Prozent.

Lea

17:52 · Reto Hess «Machen wir.»

Der Stack läuft auf deinem Laptop. Er bleibt da.

AHA 1

Flugmodus

AHA 2

JPEG für Gewichte

AHA 3

Eine Zeile Code

AHA 4

Der Raum tippt mit

AHA 5

Urlaub findet Ferien

AHA 6

Dein Stream, dein
Router

AHA 7

Ihr habt gewürfelt

Repo · scannen



github.com/dpinezich/workshop-local-first-ai

Branch `loesung` = alles gelöst.

Alles in `docs/`

Cheat-Sheet, Glossar, die Modell-Landkarte, und **alle Folien von heute als PDF**, dunkel und hell zum Ausdrucken. Dazu `aufraeumen.md`: was auf den Laptop kam, mit Grösse und Löschbefehl.

Probier morgen als Erstes

Ein wiederkehrender Task. Eine Woche lokal. Qualität messen. Dann hast du deinen Business Case.

LETZTE RUNDE LOKAL ODER CLOUD?

Ein letztes Wort. Nicht von mir.

qwen3.5:9b · localhost:11434

Taste A · WLAN ist aus

■

Der Laptop war den ganzen Tag hier. Jetzt verabschiedet er sich selbst.

Was heute auf deinen Laptop kam, und wie du es wieder loswirst

WAS	WO	GRÖSSE	ENTFERNEN
Modelle	~/.ollama/models	10-20 GB	ollama rm qwen3.5:4b qwen3.5:9b-q8_0 – qwen3.5:9b behalten
Ollama-App	Programme · Windows: Apps · Linux: /usr/local/bin	1 GB	App beenden, deinstallieren, ~/.ollama löschen
Whisper	brew · ~/whisper	2 GB	brew uninstall whisper-cpp · rm -rf ~/whisper
Repo	local-first-ai/	1 GB	Ordner löschen – nichts liegt ausserhalb. Lösungen bleiben auf GitHub
Continue	VS Code · ~/.continue	200 MB	Extension deinstallieren, Ordner löschen

Empfehlung: Ollama und **ein** Modell behalten – 7 GB, und der Task von morgen läuft. Python, Node, VS Code lassen wir in Ruhe.

SEIT 9:14

WLAN wieder ein.

Hat es jemand vermisst?



Danke.

David Pinezich · apigenio.ch



LinkedIn

linkedin.com/in/david-pinezich

Mail

david.pinezich@apigenio.ch

Telefon

+41 77 267 15 96

Web

apigenio.ch

Modelle vom Instruktor kopieren

1 • Ordner holen

```
# Vom Instruktor:  
# Ordner ~/.ollama/models  
# per AirDrop oder USB-Stick (7-11 GB)  
  
# Mac: AirDrop annehmen → landet in ~/Downloads/models  
# Windows / Linux: USB-Stick, Ordner models
```

2 • In den eigenen Ordner mergen

```
# Mac / Linux  
cp -R ~/Downloads/models/* ~/.ollama/models/  
  
# Windows  
xcopy /E /I E:\models %USERPROFILE%\ollama\models  
  
# dann Ollama neu starten  
ollama list
```

Whisper genauso: Ordner `~/whisper` vom Instruktor (Folie 57).